

第五章 Socket编程（面试突击★★★★★）

Socket编程是后台开发面试的必考内容，尤其是IO多路复用（epoll）和Reactor模型，属于高频考点。

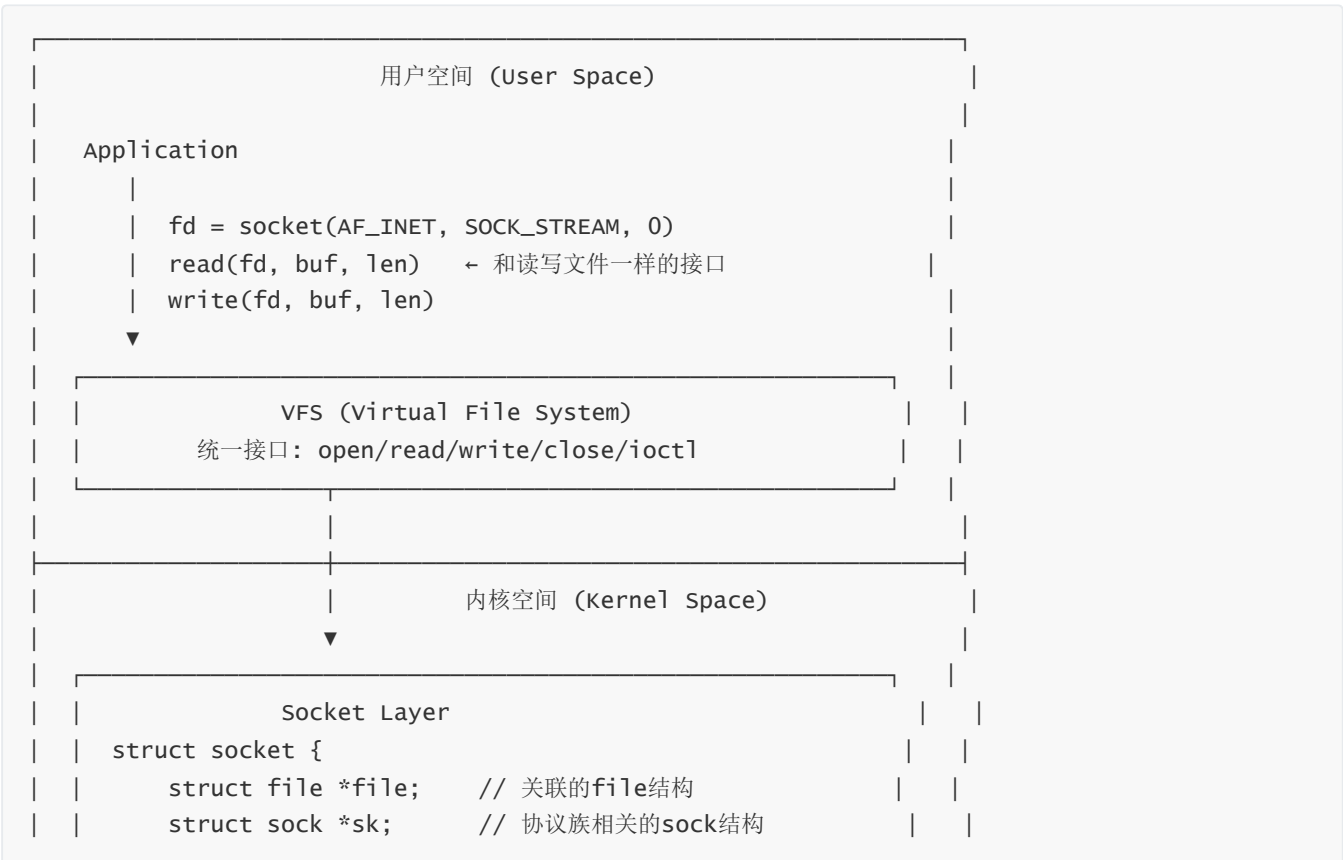
目录

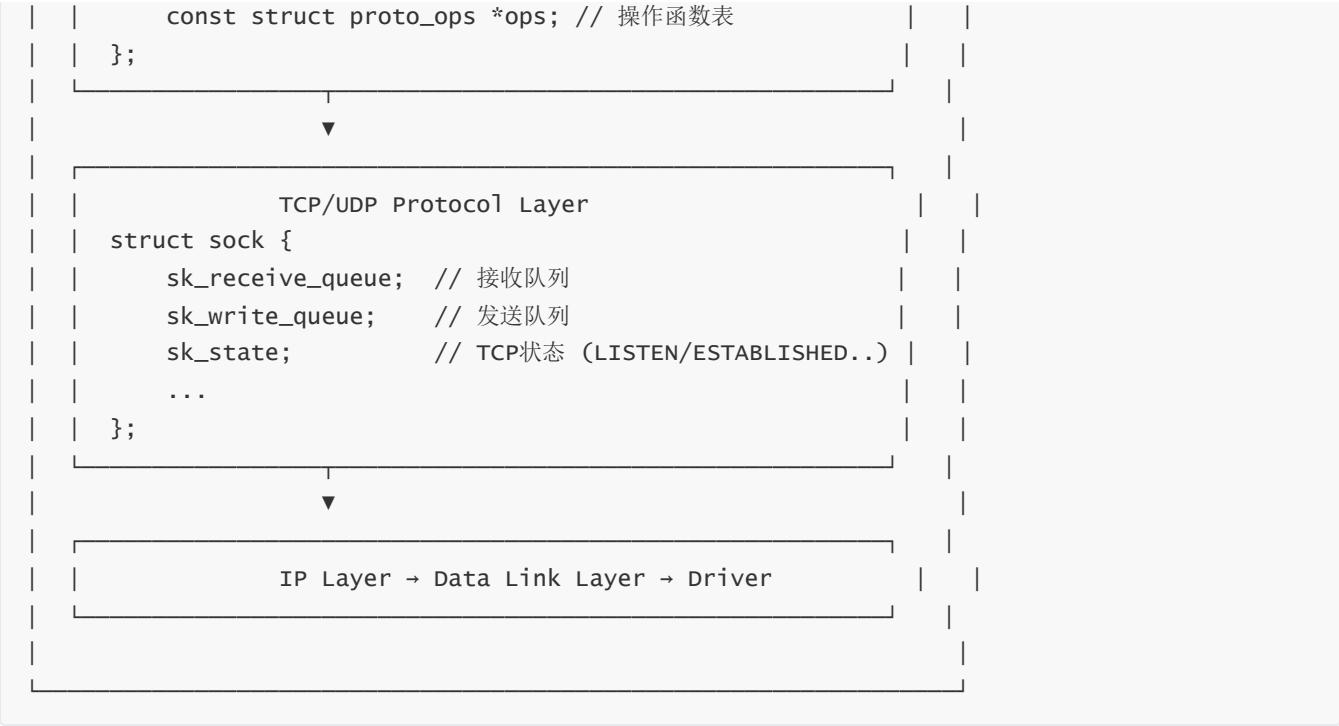
- 1. [Socket基础](#)
- 2. [Socket API详解](#)
- 3. [TCP Socket编程完整示例](#)
- 4. [IO模型](#)
- 5. [IO多路复用深度解析](#)
- 6. [Reactor模型](#)
- 7. [Proactor模型](#)
- 8. [常见Socket问题](#)
- 9. [面试总结](#)

1. Socket基础

1.1 什么是Socket

Socket（套接字）是网络通信的端点，从Linux内核的角度看，**Socket就是一个特殊的文件描述符（File Descriptor）**。





关键理解： 在Unix/Linux中，"一切皆文件"。Socket创建后会返回一个整数fd，你可以像操作文件一样对这个fd进行read/write/close操作。内核内部会通过VFS分发到Socket特定的实现。

1.2 Socket类型

类型	宏定义	协议	特点
流式Socket	SOCK_STREAM	TCP	面向连接、可靠、有序、字节流、无边界
数据报Socket	SOCK_DGRAM	UDP	无连接、不可靠、保留边界、最大长度限制
原始Socket	SOCK_RAW	直接IP	绕过传输层、可自定义协议头、需要root权限

```
// TCP Socket - 可靠的字节流
int tcp_sock = socket(AF_INET, SOCK_STREAM, 0);
// 特点：三次握手建立连接，有序传输，有流量控制和拥塞控制
// 数据没有边界——发送10次，每次100字节，对方可能一次收到1000字节

// UDP Socket - 不可靠的数据报
int udp_sock = socket(AF_INET, SOCK_DGRAM, 0);
// 特点：无连接，直接发送，保留消息边界
// 发送10次，每次100字节，对方recvfrom也是10次，每次100字节

// Raw Socket - 原始套接字（通常需要root权限）
int raw_sock = socket(AF_INET, SOCK_RAW, IPPROTO_TCP);
// 用途：抓包、自定义协议、实现ping等
```

TCP字节流 vs UDP数据报的边界问题（高频考点）：

TCP 字节流（无边界）：

发送端：write 100字节 | write 200字节 | write 150字节

↓ TCP字节流合并/拆分 ↓

接收端：read可能一次收到87字节，再收213字节，再收150字节

→ 需要在应用层自己处理分包（长度前缀、分隔符等）

UDP 数据报（有边界）：

发送端：sendto 100字节 | sendto 200字节 | sendto 150字节

↓ 独立数据报 ↓

接收端：recvfrom收到100字节 | recvfrom收到200字节 | recvfrom收到150字节

→ 每次收到的都是一个完整的报文，不会合并或拆分

1.3 Socket地址结构

sockaddr_in (IPv4)

```
#include <netinet/in.h>

struct sockaddr_in {
    sa_family_t    sin_family; // 地址族：AF_INET
    in_port_t      sin_port;   // 端口号：网络字节序（大端）
    struct in_addr sin_addr;    // IP地址：32位
    char           sin_zero[8]; // 填充字节，必须置零
};

struct in_addr {
    uint32_t s_addr; // 32位IPv4地址，网络字节序
};
```

sockaddr_in6 (IPv6)

```
struct sockaddr_in6 {
    sa_family_t    sin6_family; // AF_INET6
    in_port_t      sin6_port;    // 端口号：网络字节序
    uint32_t       sin6_flowinfo; // 流信息
    struct in6_addr sin6_addr;    // IPv6地址：128位
    uint32_t       sin6_scope_id; // scope ID
};

struct in6_addr {
    unsigned char s6_addr[16]; // 128位IPv6地址
};
```

通用sockaddr

```
// 通用地址结构，所有socket函数使用
struct sockaddr {
    sa_family_t sa_family;    // 地址族
    char        sa_data[14]; // 协议地址（可变长度）
};

// 实际使用时需要强制类型转换
struct sockaddr_in addr;
bind(sockfd, (struct sockaddr*)&addr, sizeof(addr));
```

addrinfo (协议无关编程)

```
struct addrinfo {
    int          ai_flags;      // AI_PASSIVE, AI_CANONNAME等
    int          ai_family;    // AF_INET, AF_INET6, AF_UNSPEC
    int          ai_socktype;   // SOCK_STREAM, SOCK_DGRAM
    int          ai_protocol;   // 0
    socklen_t    ai_addrlen;    // ai_addr的长度
    struct sockaddr *ai_addr;    // 指向sockaddr的指针
    char         *ai_canonname; // 规范名
    struct addrinfo *ai_next;    // 链表的下一个节点
};

// getaddrinfo - 协议无关的地址解析
// 替代传统的 gethostbyname，支持IPv4/IPv6双栈
int getaddrinfo(const char *node,    // 主机名或IP字符串
               const char *service, // 端口号或服务名
               const struct addrinfo *hints,
               struct addrinfo **res);
```

1.4 字节序转换

```
// 网络字节序 = 大端 (Big Endian)
// x86/ARM等大多数CPU = 小端 (Little Endian)

#include <arpa/inet.h>

uint32_t htonl(uint32_t hostlong);    // Host to Network Long (32位)
uint16_t htons(uint16_t hostshort);   // Host to Network Short (16位)
uint32_t ntohl(uint32_t netlong);     // Network to Host Long
uint16_t ntohs(uint16_t netshort);    // Network to Host Short

// IP地址转换
int inet_pton(int af, const char *src, void *dst);
const char *inet_ntop(int af, const void *src, char *dst, socklen_t size);

// 弃用的旧函数（不要用）
int inet_aton(const char *cp, struct in_addr *inp);
char *inet_ntoa(struct in_addr in);
```

2. Socket API详解

2.1 socket() - 创建Socket

```
#include <sys/types.h>
#include <sys/socket.h>

int socket(int domain, int type, int protocol);

// 参数:
//  domain:    AF_INET(IPv4), AF_INET6(IPv6), AF_UNIX(本地)
//  type:      SOCK_STREAM(TCP), SOCK_DGRAM(UDP), SOCK_RAW
//  protocol: 通常为0 (自动选择), 或IPPROTO_TCP, IPPROTO_UDP
// 返回值:
//  成功: 文件描述符 (>=0)
//  失败: -1, errno被设置
```

```
// 完整示例: 创建socket
#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>
#include <netinet/in.h>

int create_tcp_socket() {
    int sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd < 0) {
        perror("socket creation failed");
        exit(EXIT_FAILURE);
    }
    printf("Socket created with fd: %d\n", sockfd);
    return sockfd;
}
```

2.2 bind() - 绑定地址

```
int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);

// 将socket绑定到特定的IP地址和端口
// 服务器端必须调用bind指定监听端口
// 客户端通常不调用bind, 由内核自动分配端口
```

```
// 完整示例: bind
int bind_socket(int sockfd, int port) {
    struct sockaddr_in addr;
    memset(&addr, 0, sizeof(addr));
    addr.sin_family      = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY; // 绑定所有网卡IP
    addr.sin_port        = htons(port); // 网络字节序
```

```

if (bind(sockfd, (struct sockaddr*)&addr, sizeof(addr)) < 0) {
    perror("bind failed");
    close(sockfd);
    exit(EXIT_FAILURE);
}
printf("Bound to port %d\n", port);
return 0;
}

```

2.3 listen() - 监听★★★

```

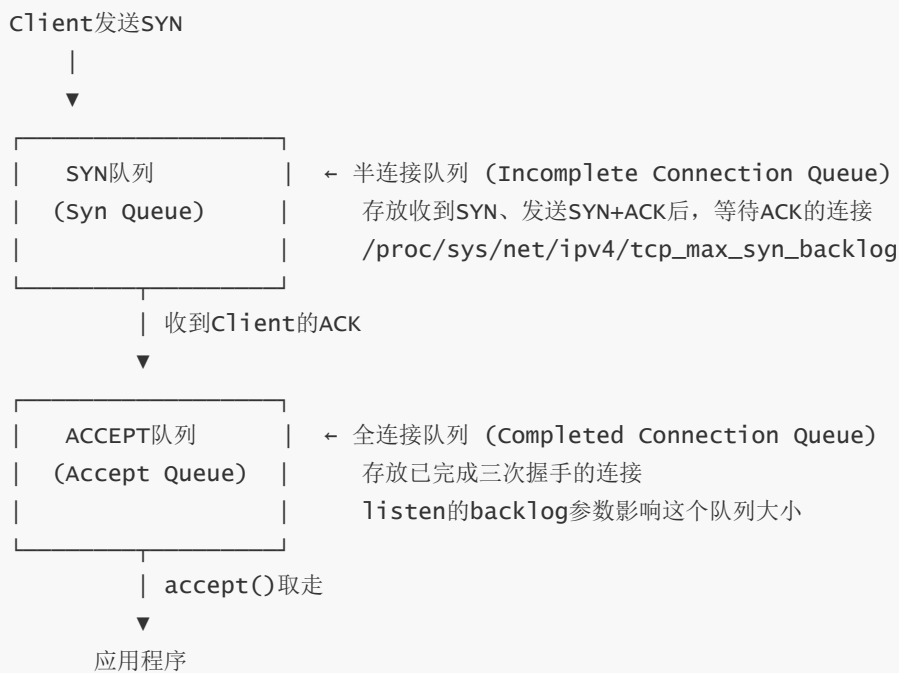
int listen(int sockfd, int backlog);

// 将主动socket转为被动socket (LISTEN状态)
// backlog: 已完成连接队列的最大长度

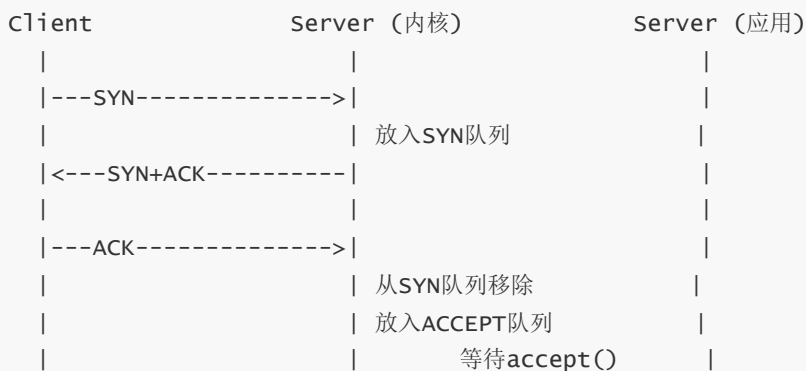
```

backlog深度解析（高频面试题）：

在内核中，TCP的连接建立分为两个队列：



TCP三次握手与两个队列的关系：



	<---accept()取走-----	
	从ACCEPT队列移除	
	连接建立(ESTABLISHED)	

实际backlog值:

- 内核参数 `net.core.somaxconn` 限制了backlog的最大值 (默认128)
- 调用 `listen(fd, backlog)` 时, 实际值 = $\min(\text{backlog}, \text{somaxconn})$
- Linux 5.4+ 中backlog不再受somaxconn限制 (默认4096)
- 对于高并发服务器, 建议设置 `somaxconn=65535`

```
// 设置较大的backlog用于高并发场景
// 同时需要设置系统参数:
// echo 65535 > /proc/sys/net/core/somaxconn
int listen_socket(int sockfd, int backlog) {
    if (listen(sockfd, backlog) < 0) {
        perror("listen failed");
        return -1;
    }
    printf("Listening with backlog=%d\n", backlog);
    return 0;
}
```

2.4 accept() - 接受连接

```
int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);

// sockfd: 监听socket (listening socket)
// addr: 获取客户端地址 (可为NULL)
// addrlen: 传入传出参数 (调用前初始化为sizeof(addr))
// 返回: 新的socket fd (connected socket), 失败返回-1

// accept是阻塞的, 当accept队列为空时阻塞等待
// 返回的新fd代表与客户端的连接, 原监听fd继续用于接受新连接
```

```
// 完整示例: accept
int accept_connection(int listen_fd) {
    struct sockaddr_in client_addr;
    socklen_t addr_len = sizeof(client_addr);
    char client_ip[INET_ADDRSTRLEN];

    int client_fd = accept(listen_fd,
                           (struct sockaddr*)&client_addr,
                           &addr_len);

    if (client_fd < 0) {
        perror("accept failed");
        return -1;
    }

    inet_ntop(AF_INET, &client_addr.sin_addr, client_ip, sizeof(client_ip));
```

```

printf("New connection: %s:%d (fd=%d)\n",
      client_ip, ntohs(client_addr.sin_port), client_fd);

return client_fd;
}

```

2.5 connect() - 连接服务器

```

int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);

```

```

// 客户端发起连接请求，触发三次握手
// 阻塞模式下，connect会阻塞直到连接建立或出错
// 成功返回0，失败返回-1

```

```

// 完整示例：connect
int connect_to_server(const char *ip, int port) {
    int sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd < 0) {
        perror("socket");
        return -1;
    }

    struct sockaddr_in server_addr;
    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(port);

    if (inet_pton(AF_INET, ip, &server_addr.sin_addr) <= 0) {
        perror("inet_pton");
        close(sockfd);
        return -1;
    }

    if (connect(sockfd, (struct sockaddr*)&server_addr,
               sizeof(server_addr)) < 0) {
        perror("connect");
        close(sockfd);
        return -1;
    }

    printf("Connected to %s:%d\n", ip, port);
    return sockfd;
}

```

2.6 recv()/send() - 收发数据

```

ssize_t recv(int sockfd, void *buf, size_t len, int flags);
ssize_t send(int sockfd, const void *buf, size_t len, int flags);

```

```

// 返回值：
//   >0: 实际读取/发送的字节数

```

```
// 0: 对端关闭连接 (recv返回0表示FIN)
// -1: 错误, 检查errno
// EAGAIN/EWOULDBLOCK: 非阻塞模式下暂无数据
// EINTR: 被信号中断
// ECONNRESET: 连接被重置

// flags常用值:
// MSG_DONTWAIT: 本次操作非阻塞 (不影响socket属性)
// MSG_PEEK: 查看数据但不从缓冲区移除 (recv专用)
// MSG_WAITALL: 等待直到收到len字节 (recv专用, 可能被信号中断)
// MSG_NOSIGNAL: 不发送SIGPIPE信号 (send专用)
```

2.7 read()/write() - 也可用于Socket

```
// read/write是文件IO接口, 也可用于socket
// 等价于 recv(fd, buf, len, 0) / send(fd, buf, len, 0)
ssize_t read(int fd, void *buf, size_t count);
ssize_t write(int fd, const void *buf, size_t count);
```

2.8 close() vs shutdown() ★★★

```
int close(int fd);
int shutdown(int sockfd, int how);
// how: SHUT_RD(0) 关闭读, SHUT_WR(1) 关闭写, SHUT_RDWR(2) 关闭读写
```

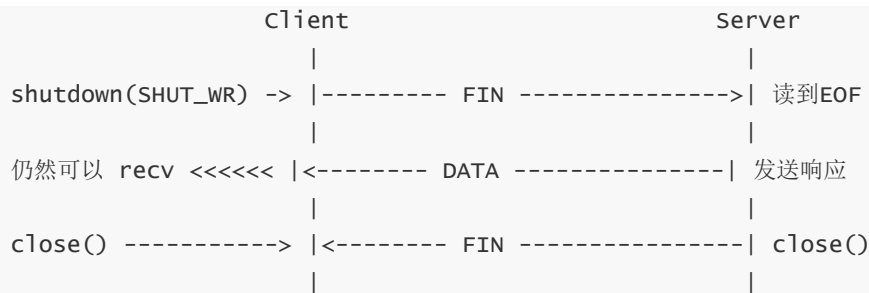
核心区别 (面试高频题) :

	close()	shutdown()
作用对象	文件描述符	Socket连接
引用计数	减少引用计数 计数为0才关闭	无视引用计数 立即关闭半连接
dup/fork	close后其他fd仍可用	影响所有共享的fd
半关闭	不支持	支持(单独关闭读/写通道)
行为	全双工关闭 释放资源	SHUT_WR: 发送FIN后仍可读 SHUT_RD: 丢弃接收缓冲 SHUT_RDWR: 全关闭

为什么需要shutdown?

场景: fork()后父子进程共享socket, fd引用计数=2
close()只减少引用计数, 实际上socket并未关闭
shutdown()无视引用计数, 直接关闭连接

场景: HTTP协议需要半关闭
客户端发送完请求后, 调用shutdown(SHUT_WR)发送FIN
服务端收到FIN后返回响应, 然后close
客户端仍然可以recv来读取响应数据



```
// shutdown示例：HTTP客户端半关闭模式
void http_client_example(const char *host, int port) {
    int sock = connect_to_server(host, port);

    // 发送HTTP请求
    char request[] = "GET / HTTP/1.1\r\nHost: example.com\r\n\r\n";
    send(sock, request, strlen(request), 0);

    // 半关闭写端 → 发送FIN，告诉服务器请求发送完毕
    // 但仍然可以接收服务器的响应
    shutdown(sock, SHUT_WR);

    // 读取服务器响应
    char buf[4096];
    ssize_t n;
    while ((n = recv(sock, buf, sizeof(buf), 0)) > 0) {
        fwrite(buf, 1, n, stdout);
    }
    // recv返回0，服务器也关闭了连接

    close(sock); // 完全关闭
}
```

2.9 getsockopt()/setsockopt() - Socket选项

```
int getsockopt(int sockfd, int level, int optname,
               void *optval, socklen_t *optlen);
int setsockopt(int sockfd, int level, int optname,
               const void *optval, socklen_t optlen);
```

常用Socket选项一览：

选项	级别	说明
SO_REUSEADDR	SOL_SOCKET	重用TIME_WAIT状态的地址
SO_REUSEPORT	SOL_SOCKET	多进程/线程绑定同一端口
SO_KEEPALIVE	SOL_SOCKET	TCP保活机制
SO_LINGER	SOL_SOCKET	close时的行为

选项	级别	说明
SO_RCVBUF	SOL_SOCKET	接收缓冲区大小
SO_SNDBUF	SOL_SOCKET	发送缓冲区大小
SO_RCVTIMEO	SOL_SOCKET	接收超时
SO_SNDTIMEO	SOL_SOCKET	发送超时
TCP_NODELAY	IPPROTO_TCP	禁用Nagle算法
TCP_CORK	IPPROTO_TCP	积累数据后发送

```
// 常用选项设置示例
void set_socket_options(int sockfd) {
    // 1. 重用地址（服务器必须设置，避免重启时TIME_WAIT导致bind失败）
    int reuse = 1;
    setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &reuse, sizeof(reuse));

    // 2. 禁用Nagle算法（低延迟场景必须）
    int nodelay = 1;
    setsockopt(sockfd, IPPROTO_TCP, TCP_NODELAY, &nodelay, sizeof(nodelay));

    // 3. 设置接收超时 5秒
    struct timeval tv = {5, 0};
    setsockopt(sockfd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv));

    // 4. 设置缓冲区大小
    int bufsize = 256 * 1024; // 256KB
    setsockopt(sockfd, SOL_SOCKET, SO_RCVBUF, &bufsize, sizeof(bufsize));
    setsockopt(sockfd, SOL_SOCKET, SO_SNDBUF, &bufsize, sizeof(bufsize));
}
```

3. TCP Socket编程完整示例

3.1 完整TCP回显服务器（迭代版）

```
/*
 * tcp_echo_server.c - 迭代式TCP回显服务器
 * 编译: gcc -o server tcp_echo_server.c
 * 运行: ./server 8888
 * 功能: 接受客户端连接，接收数据并原样返回，处理完后关闭连接
 *
 * 注意: 这是迭代式服务器，同一时刻只能处理一个客户端
 *       生产环境应使用多进程/多线程/IO多路复用
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```

#include <unistd.h>
#include <errno.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <netinet/tcp.h>
#include <arpa/inet.h>
#include <signal.h>

#define BUFFER_SIZE 4096
#define BACKLOG 128

// 信号处理：忽略SIGPIPE（向已关闭的socket写数据会触发）
void setup_signal_handler() {
    signal(SIGPIPE, SIG_IGN); // 忽略SIGPIPE，让write返回EPIPE错误
}

// 创建并绑定监听socket
int create_listen_socket(int port) {
    int listen_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (listen_fd < 0) {
        perror("socket");
        return -1;
    }

    // 设置SO_REUSEADDR：允许重用TIME_WAIT状态的端口
    int optval = 1;
    if (setsockopt(listen_fd, SOL_SOCKET, SO_REUSEADDR,
                    &optval, sizeof(optval)) < 0) {
        perror("setsockopt SO_REUSEADDR");
        close(listen_fd);
        return -1;
    }

    struct sockaddr_in addr;
    memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY; // 监听所有网卡
    addr.sin_port = htons(port);

    if (bind(listen_fd, (struct sockaddr*)&addr, sizeof(addr)) < 0) {
        perror("bind");
        close(listen_fd);
        return -1;
    }

    if (listen(listen_fd, BACKLOG) < 0) {
        perror("listen");
        close(listen_fd);
        return -1;
    }

    return listen_fd;
}

```

```

}

// 处理单个客户端--回显数据
void handle_client(int client_fd) {
    char buffer[BUFFER_SIZE];
    ssize_t n;

    // 循环接收，直到客户端关闭连接（recv返回0）
    while ((n = recv(client_fd, buffer, sizeof(buffer), 0)) > 0) {
        // 回显：将收到的数据原样发回
        ssize_t sent = 0;
        while (sent < n) {
            ssize_t ret = send(client_fd, buffer + sent, n - sent, 0);
            if (ret < 0) {
                if (errno == EINTR) continue; // 被信号中断，重试
                perror("send");
                goto cleanup;
            }
            sent += ret;
        }
    }

    if (n < 0) {
        perror("recv");
    } else {
        printf("Client closed connection (recv returned 0)\n");
    }

cleanup:
    close(client_fd);
}

// 辅助函数：打印客户端信息
void print_client_info(struct sockaddr_in *addr) {
    char ip[INET_ADDRSTRLEN];
    inet_ntop(AF_INET, &addr->sin_addr, ip, sizeof(ip));
    printf("[Connection] %s:%d\n", ip, ntohs(addr->sin_port));
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <port>\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    int port = atoi(argv[1]);
    setup_signal_handler();

    int listen_fd = create_listen_socket(port);
    if (listen_fd < 0) {
        exit(EXIT_FAILURE);
    }
}

```

```

printf("TCP Echo Server listening on port %d\n", port);

// 主循环：接受连接并处理
while (1) {
    struct sockaddr_in client_addr;
    socklen_t addr_len = sizeof(client_addr);

    int client_fd = accept(listen_fd,
                           (struct sockaddr*)&client_addr,
                           &addr_len);

    if (client_fd < 0) {
        if (errno == EINTR) continue; // 被信号中断，继续
        perror("accept");
        break;
    }

    print_client_info(&client_addr);
    handle_client(client_fd);
    // client_fd在handle_client中被close
}

close(listen_fd);
return 0;
}

```

3.2 完整TCP客户端

```

/*
 * tcp_echo_client.c - TCP回显客户端
 * 编译: gcc -o client tcp_echo_client.c
 * 运行: ./client 127.0.0.1 8888
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define BUFFER_SIZE 4096

int connect_to_server(const char *server_ip, int server_port) {
    int sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd < 0) {
        perror("socket");
        return -1;
    }

    struct sockaddr_in addr;

```

```

memset(&addr, 0, sizeof(addr));
addr.sin_family = AF_INET;
addr.sin_port = htons(server_port);

if (inet_pton(AF_INET, server_ip, &addr.sin_addr) <= 0) {
    fprintf(stderr, "Invalid IP address: %s\n", server_ip);
    close(sockfd);
    return -1;
}

if (connect(sockfd, (struct sockaddr*)&addr, sizeof(addr)) < 0) {
    perror("connect");
    close(sockfd);
    return -1;
}

printf("Connected to %s:%d\n", server_ip, server_port);
return sockfd;
}

int main(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "Usage: %s <server_ip> <port>\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    const char *server_ip = argv[1];
    int server_port = atoi(argv[2]);

    int sockfd = connect_to_server(server_ip, server_port);
    if (sockfd < 0) exit(EXIT_FAILURE);

    char send_buf[BUFFER_SIZE];
    char recv_buf[BUFFER_SIZE];

    printf("Enter messages (Ctrl+D to quit):\n");

    while (fgets(send_buf, sizeof(send_buf), stdin) != NULL) {
        size_t len = strlen(send_buf);

        // 发送数据
        ssize_t sent = send(sockfd, send_buf, len, 0);
        if (sent < 0) {
            perror("send");
            break;
        }

        // 接收回显
        ssize_t total = 0;
        while (total < len) {
            ssize_t n = recv(sockfd, recv_buf + total,
                             sizeof(recv_buf) - total, 0);
            if (n <= 0) {

```

```

        if (n == 0) printf("Server closed connection\n");
        else perror("recv");
        goto done;
    }
    total += n;
}

printf("Echo: %.*s", (int)total, recv_buf);
}

done:
    close(sockfd);
    return 0;
}

```

3.3 多进程并发服务器 (fork)

```

/*
 * tcp_fork_server.c - fork多进程并发TCP服务器
 * 编译: gcc -o fork_server tcp_fork_server.c
 *
 * 工作原理:
 *   主进程accept连接后fork子进程处理
 *   子进程继承client_fd, 处理完毕后exit
 *   主进程立即close(client_fd)并继续accept (子进程持有fd, 主进程不需要)
 *
 * 优点: 简单, 进程间天然隔离
 * 缺点: fork开销大, 进程间通信不便, 大量子进程导致系统负担
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <signal.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define BUFFER_SIZE 4096
#define BACKLOG     128

// SIGCHLD处理函数: 回收僵尸进程
void sigchld_handler(int sig) {
    // waitpid使用WNOHANG, 因为多个子进程可能同时退出
    // 需要用循环回收所有已退出的子进程
    int saved_errno = errno;
    while (waitpid(-1, NULL, WNOHANG) > 0)
        ;
}

```

```

    errno = saved_errno;
}

void handle_client(int client_fd, struct sockaddr_in *client_addr) {
    char ip[INET_ADDRSTRLEN];
    inet_ntop(AF_INET, &client_addr->sin_addr, ip, sizeof(ip));
    printf("[Child %d] Handling client %s:%d\n",
           getpid(), ip, ntohs(client_addr->sin_port));

    char buffer[BUFFER_SIZE];

    while (1) {
        ssize_t n = recv(client_fd, buffer, sizeof(buffer), 0);
        if (n <= 0) {
            if (n == 0)
                printf("[Child %d] client closed connection\n", getpid());
            else
                perror("recv");
            break;
        }

        // 回显
        if (send(client_fd, buffer, n, 0) < 0) {
            perror("send");
            break;
        }
    }

    close(client_fd);
    printf("[Child %d] Done\n", getpid());
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <port>\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    int port = atoi(argv[1]);

    // 安装SIGCHLD处理, 避免僵尸进程
    struct sigaction sa;
    sa.sa_handler = sigchld_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART;
    sigaction(SIGCHLD, &sa, NULL);

    // 忽略SIGPIPE
    signal(SIGPIPE, SIG_IGN);

    // 创建监听socket
    int listen_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (listen_fd < 0) {

```

```

    perror("socket");
    exit(EXIT_FAILURE);
}

int optval = 1;
setsockopt(listen_fd, SOL_SOCKET, SO_REUSEADDR, &optval, sizeof(optval));

struct sockaddr_in server_addr;
memset(&server_addr, 0, sizeof(server_addr));
server_addr.sin_family = AF_INET;
server_addr.sin_addr.s_addr = INADDR_ANY;
server_addr.sin_port = htons(port);

if (bind(listen_fd, (struct sockaddr*)&server_addr,
        sizeof(server_addr)) < 0) {
    perror("bind");
    exit(EXIT_FAILURE);
}

if (listen(listen_fd, BACKLOG) < 0) {
    perror("listen");
    exit(EXIT_FAILURE);
}

printf("Fork-based TCP server listening on port %d (PID=%d)\n",
        port, getpid());

while (1) {
    struct sockaddr_in client_addr;
    socklen_t addr_len = sizeof(client_addr);

    int client_fd = accept(listen_fd,
                           (struct sockaddr*)&client_addr,
                           &addr_len);

    if (client_fd < 0) {
        if (errno == EINTR) continue;
        perror("accept");
        break;
    }

    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        close(client_fd);
        continue;
    } else if (pid == 0) {
        // 子进程
        close(listen_fd); // 子进程不需要监听socket
        handle_client(client_fd, &client_addr);
        exit(EXIT_SUCCESS);
    } else {
        // 父进程
        // 重要：父进程必须close(client_fd)，否则：

```

```

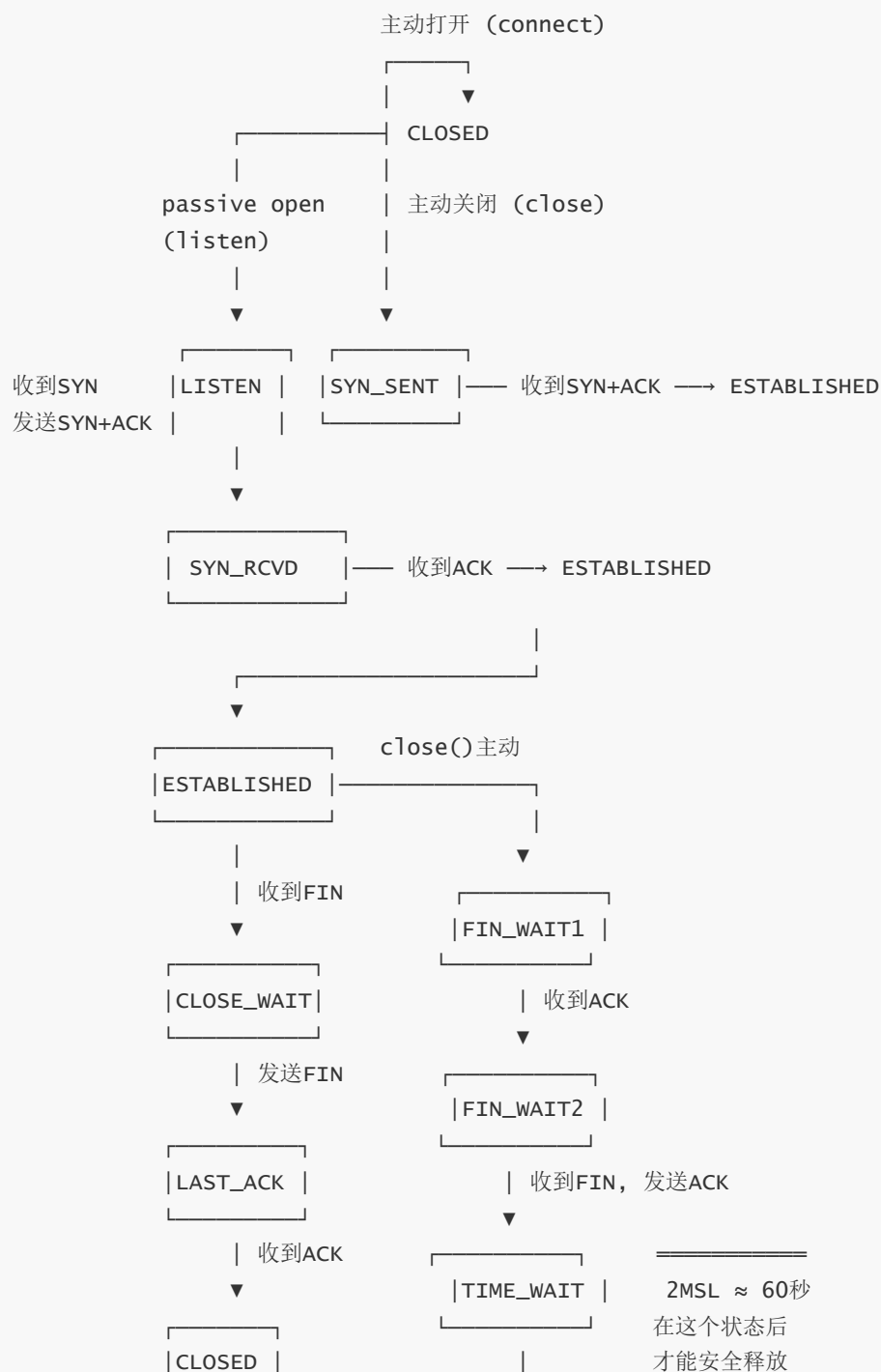
// 1. 文件描述符泄漏
// 2. 客户端关闭后，子进程recv不会返回0（引用计数未归零）
close(client_fd);
}
}

close(listen_fd);
return 0;
}

```

3.4 TCP状态转换与netstat观察

TCP状态转换图（核心，必须背诵）：



端口

CLOSED

TIME_WAIT详解（高频考点）：

- 主动关闭方进入TIME_WAIT状态
- 持续2MSL（Maximum Segment Lifetime，通常60秒）
- 两个原因：
 1. 确保最后一个ACK能到达对端（如果ACK丢失，对端重发FIN）
 2. 让旧连接的残留报文在网络中消失
- 服务器端出现大量TIME_WAIT：
原因：服务器主动关闭连接
解决：SO_REUSEADDR、调整tcp_tw_reuse、连接池、长连接

使用netstat观察TCP状态：

```
# 查看所有TCP连接
netstat -antp

# 查看特定端口
netstat -antp | grep 8888

# 统计各状态连接数
netstat -ant | awk '{print $6}' | sort | uniq -c | sort -rn

# 观察TIME_WAIT（通常是主动关闭方）
netstat -ant | grep TIME_WAIT | wc -l

# 查看socket统计
ss -s

# 查看某个进程的socket
ss -antp | grep <pid>
```

4. IO模型

Unix/Linux下共有5种IO模型。以"从socket读取数据"为例，整个过程分为两个阶段：

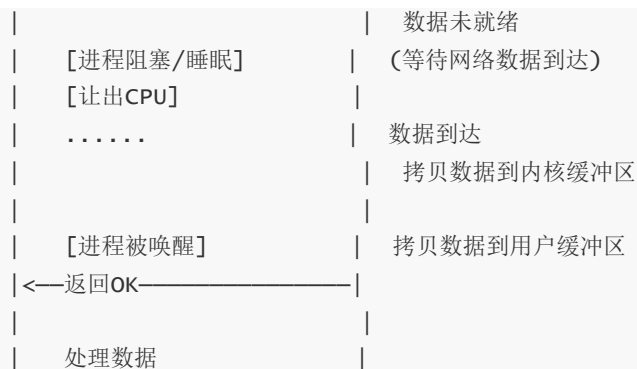
1. **等待数据**：数据从网络到达内核缓冲区
2. **拷贝数据**：数据从内核缓冲区拷贝到用户缓冲区

4.1 阻塞IO (Blocking IO) ★

定义： 应用进程调用recv，如果内核数据未就绪，进程进入阻塞状态（睡眠），让出CPU，直到数据就绪被唤醒。

阻塞IO流程：

应用进程	内核
—recv()—→	



代码示例：

```
// 阻塞IO - 默认行为
int blocking_recv(int fd) {
    char buf[1024];
    // 如果内核接收缓冲区为空，recv会阻塞
    // 进程进入TASK_INTERRUPTIBLE状态，不可运行
    ssize_t n = recv(fd, buf, sizeof(buf), 0);
    if (n > 0) {
        // 处理数据
    }
    return n;
}
```

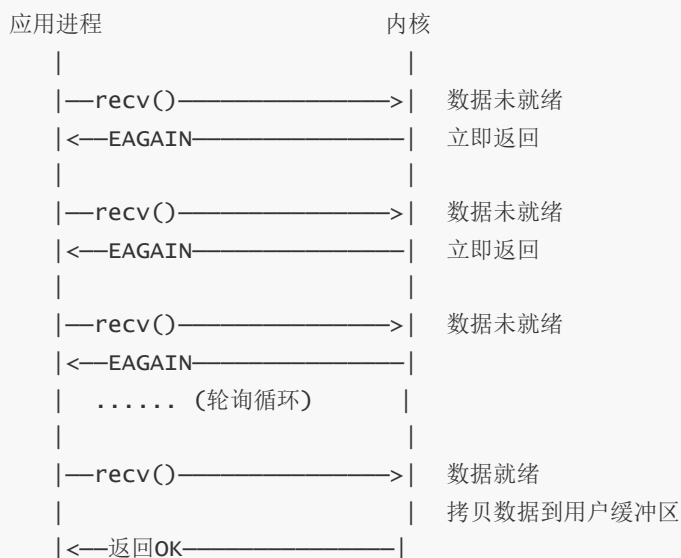
优点： 编程简单，进程在等待IO时不消耗CPU

缺点： 每个连接需要一个线程/进程，大量连接时资源消耗大；处理IO的线程大部分时间在睡眠，效率低

4.2 非阻塞IO (Non-blocking IO)

定义： 设置socket为非阻塞模式。如果数据未就绪，recv立即返回-1，errno=EAGAIN/EWOULDBLOCK。应用需要反复轮询直到数据就绪。

非阻塞IO流程：



代码示例：

```

#include <fcntl.h>

// 设置socket为非阻塞
int set_nonblocking(int fd) {
    int flags = fcntl(fd, F_GETFL, 0);
    if (flags < 0) {
        perror("fcntl F_GETFL");
        return -1;
    }
    if (fcntl(fd, F_SETFL, flags | O_NONBLOCK) < 0) {
        perror("fcntl F_SETFL");
        return -1;
    }
    return 0;
}

// 非阻塞IO读取——轮询模式
int nonblocking_recv(int fd) {
    char buf[1024];
    while (1) {
        ssize_t n = recv(fd, buf, sizeof(buf), 0);
        if (n > 0) {
            return n; // 有数据
        } else if (n < 0) {
            if (errno == EAGAIN || errno == EWOULDBLOCK) {
                // 暂无数据，可以在这里做其他事情
                // 或用usleep避免忙等
                usleep(1000); // 1ms
                continue;
            }
            perror("recv");
            return -1;
        } else {
            return 0; // 连接关闭
        }
    }
}

```

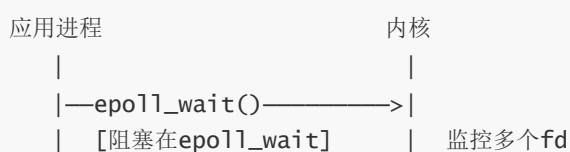
优点： 一个线程可处理多个连接

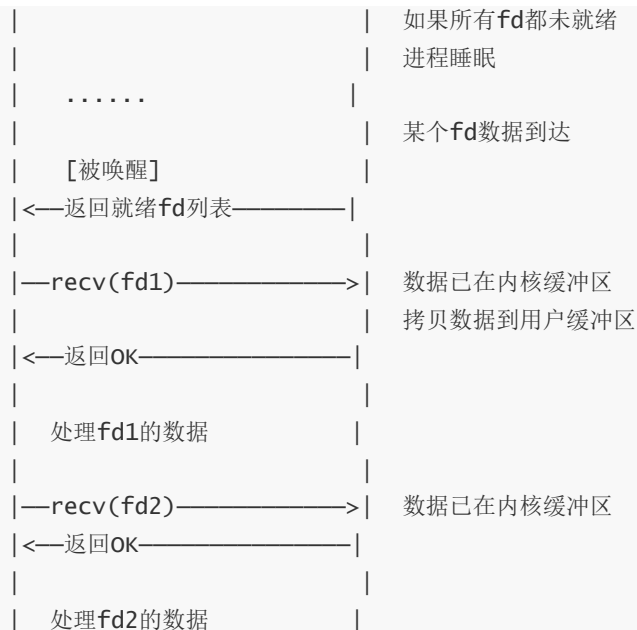
缺点： 轮询浪费CPU，数据不及时

4.3 IO多路复用 (IO Multiplexing) ★★★

定义： 使用select/poll/epoll同时监控多个fd，当任一fd就绪时通知应用进程。进程阻塞在select/poll/epoll上，而不是阻塞在recv上。

IO多路复用流程：





- 优点：** 一个线程可管理大量连接，不是轮询而是事件驱动
- 缺点：** 需要两次系统调用（epoll_wait + recv），有上下文切换开销

4.4 信号驱动IO (Signal-driven IO)

定义： 使用sigaction注册SIGIO信号处理函数。数据就绪时内核发送SIGIO信号，在信号处理函数中读取数据。

信号驱动IO流程：



代码示例：

```
#include <signal.h>

int sock_fd; // 全局变量，信号处理函数中使用

void sigio_handler(int sig) {
    char buf[1024];
    // 在信号处理函数中读取数据
    ssize_t n = recv(sock_fd, buf, sizeof(buf), 0);
    if (n > 0) {
```

```

        printf("Received %zd bytes\n", n);
    }
}

void setup_signal_driven_io(int fd) {
    sock_fd = fd;

    // 注册SIGIO处理函数
    struct sigaction sa;
    sa.sa_handler = sigio_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART;
    sigaction(SIGIO, &sa, NULL);

    // 设置接收SIGIO的进程（必须是当前进程）
    fcntl(fd, F_SETOWN, getpid());

    // 启用异步通知模式
    int flags = fcntl(fd, F_GETFL);
    fcntl(fd, F_SETFL, flags | O_ASYNC);
}

```

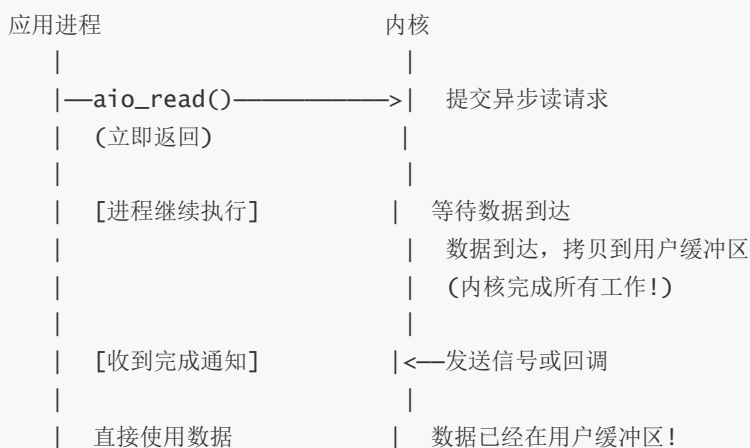
优点： 进程在数据到达前不需要等待/轮询，可以继续执行

缺点： 信号处理复杂，信号不排队（多个fd数据到达只有一次通知），不适合高并发

4.5 异步IO (Asynchronous IO, AIO)

定义： 告知内核启动IO操作，内核完成后通知进程。与信号驱动IO的区别：信号驱动IO通知"可以开始IO"，AIO通知"IO已完成"。

异步IO流程（真正的异步）：



AIO vs 信号驱动IO:

信号驱动IO：内核通知"数据准备好了，你可以读了"

→ 用户进程还需要调用recv，数据仍在内核缓冲区

异步IO：内核通知"数据已经在你的缓冲区了，用吧"

→ 用户进程不需要再调用recv，内核已经完成了拷贝

Linux AIO接口：

```
#include <aio.h>
#include <signal.h>

// AIO控制块
void aio_read_example(int fd) {
    struct aiocb cb;
    memset(&cb, 0, sizeof(cb));
    cb.aio_fildes = fd;
    cb.aio_buf = malloc(1024);
    cb.aio_nbytes = 1024;
    cb.aio_offset = 0;

    // 设置完成通知方式：信号
    cb.aio_sigevent.sigev_notify = SIGEV_SIGNAL;
    cb.aio_sigevent.sigev_signo = SIGUSR1;

    // 提交异步读请求
    if (aio_read(&cb) < 0) {
        perror("aio_read");
        return;
    }

    // 进程可以继续执行其他任务...
    // 读完成后会收到SIGUSR1信号

    // 检查是否完成
    while (aio_error(&cb) == EINPROGRESS) {
        // 做其他事情
    }

    // 获取结果
    ssize_t ret = aio_return(&cb);
    if (ret > 0) {
        printf("Read %zd bytes asynchronously\n", ret);
    }

    free((void*)cb.aio_buf);
}
```

注意：Linux的POSIX AIO实现（glibc aio）实际上是在用户空间用线程池模拟的，并非真正内核AIO。真正的内核AIO需要：

- **Linux原生AIO**（`io_submit`等）：只对O_DIRECT打开的文件有效，对socket无效
- **io_uring**（Linux 5.1+）：新一代异步IO接口，支持socket、文件等

4.6 五种IO模型对比

五种IO模型对比：

阻塞IO：[阻塞等待]→[recv拷贝]

非阻塞IO：[轮询检查]→[recv拷贝]

IO多路复用：[select阻塞]→[recv拷贝]

信号驱动IO：[不等待]→[收到信号]→[recv拷贝]

异步IO：[不等待]→[收到通知，数据已在用户缓冲区]

同步 vs 异步的判断标准：

- 第二阶段(数据拷贝)会阻塞调用线程 → 同步IO

- 第二阶段(数据拷贝)也不阻塞 → 异步IO

所以：前四种都是同步IO，只有第5种是真正的异步IO

5. IO多路复用深度解析

5.1 select()

工作原理

select内部工作原理：

用户空间

内核空间

fd_set

readfds

进程睡眠

(block)

进程醒来

遍历fd_set

FD_ISSET

找到就绪fd

select()

遍历所有fd

检查每个fd的状态

cpu A的socket未就绪

cpu B的socket未就绪

cpu C的socket未就绪

...

将所有fd加入等待队列

当任一fd就绪，唤醒进程

再次遍历所有fd

将就绪的fd在fd_set中置位

返回就绪fd数量

关键问题：select需要O(n)遍历，n是所有监控的fd数量

每次调用都要重新传入整个fd_set

返回后应用程序还要O(n)遍历找到就绪的fd

fd_set结构与FD_SETSIZE限制

```
// fd_set本质是一个位图 (bitmap)
// FD_SETSIZE 默认1024，即最多监控1024个fd
// 在 <sys/select.h> 中定义

typedef struct {
    // 典型实现: long fds_bits[FD_SETSIZE / (8 * sizeof(long))]
    // 对于1024: long fds_bits[16] (16 * 64 = 1024 bits)
    long fds_bits[FD_SETSIZE / (8 * sizeof(long))];
} fd_set;

// 操作宏
void FD_ZERO(fd_set *set);           // 清空所有位
void FD_SET(int fd, fd_set *set);    // 设置某位
void FD_CLR(int fd, fd_set *set);    // 清除某位
int  FD_ISSET(int fd, fd_set *set);  // 检查某位是否被设置
```

完整select服务器

```
/*
 * select_echo_server.c - 基于select的并发回显服务器
 * 编译: gcc -o select_server select_echo_server.c
 *
 * 特点: 单进程管理多个连接, 不再需要fork
 * 缺点: FD_SETSIZE限制 (最多1024个连接), O(n)遍历效率低
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <sys/select.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define MAX_CLIENTS FD_SETSIZE // 最多1024个连接
#define BUFFER_SIZE 4096
#define BACKLOG 128

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <port>\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    int port = atoi(argv[1]);

    // 创建监听socket
```

```

int listen_fd = socket(AF_INET, SOCK_STREAM, 0);
if (listen_fd < 0) { perror("socket"); exit(EXIT_FAILURE); }

int optval = 1;
setsockopt(listen_fd, SOL_SOCKET, SO_REUSEADDR, &optval, sizeof(optval));

struct sockaddr_in server_addr;
memset(&server_addr, 0, sizeof(server_addr));
server_addr.sin_family = AF_INET;
server_addr.sin_addr.s_addr = INADDR_ANY;
server_addr.sin_port = htons(port);

if (bind(listen_fd, (struct sockaddr*)&server_addr,
        sizeof(server_addr)) < 0) {
    perror("bind"); exit(EXIT_FAILURE);
}

if (listen(listen_fd, BACKLOG) < 0) {
    perror("listen"); exit(EXIT_FAILURE);
}

printf("Select-based server listening on port %d\n", port);

// 客户端fd数组（简化版，不使用fd_set存储所有客户端fd）
int client_fds[MAX_CLIENTS];
for (int i = 0; i < MAX_CLIENTS; i++) {
    client_fds[i] = -1; // -1表示未使用
}

fd_set readfds; // 读事件集合
int max_fd;     // 当前最大的fd值，用于select的第一个参数

while (1) {
    FD_ZERO(&readfds);

    // 添加监听socket
    FD_SET(listen_fd, &readfds);
    max_fd = listen_fd;

    // 添加所有客户端socket
    for (int i = 0; i < MAX_CLIENTS; i++) {
        if (client_fds[i] != -1) {
            FD_SET(client_fds[i], &readfds);
            if (client_fds[i] > max_fd) {
                max_fd = client_fds[i];
            }
        }
    }

    // select: 阻塞等待事件
    // 注意: select会修改readfds，所以每次循环都要重新设置
    int ready = select(max_fd + 1, &readfds, NULL, NULL, NULL);
    if (ready < 0) {

```

```

        if (errno == EINTR) continue;
        perror("select");
        break;
    }

    // 1. 检查监听socket是否有新连接
    if (FD_ISSET(listen_fd, &readfds)) {
        struct sockaddr_in client_addr;
        socklen_t addr_len = sizeof(client_addr);

        int client_fd = accept(listen_fd,
                                (struct sockaddr*)&client_addr,
                                &addr_len);

        if (client_fd < 0) {
            perror("accept");
        } else {
            // 将新客户端加入数组
            int i;
            for (i = 0; i < MAX_CLIENTS; i++) {
                if (client_fds[i] == -1) {
                    client_fds[i] = client_fd;
                    break;
                }
            }
            if (i == MAX_CLIENTS) {
                fprintf(stderr, "Too many clients, rejecting\n");
                close(client_fd);
            } else {
                char ip[INET_ADDRSTRLEN];
                inet_ntop(AF_INET, &client_addr.sin_addr,
                          ip, sizeof(ip));
                printf("New client: %s:%d (fd=%d, slot=%d)\n",
                      ip, ntohs(client_addr.sin_port), client_fd, i);
            }
        }
    }
}

// 2. 遍历所有客户端, 检查是否有数据可读
for (int i = 0; i < MAX_CLIENTS; i++) {
    int fd = client_fds[i];
    if (fd == -1) continue;

    if (FD_ISSET(fd, &readfds)) {
        char buffer[BUFFER_SIZE];
        ssize_t n = recv(fd, buffer, sizeof(buffer), 0);

        if (n <= 0) {
            if (n == 0) {
                printf("Client fd=%d disconnected\n", fd);
            } else {
                perror("recv");
            }
            close(fd);
        }
    }
}

```

```

        client_fds[i] = -1;
    } else {
        // 回显
        send(fd, buffer, n, 0);
    }
}
}

close(listen_fd);
return 0;
}

```

select性能分析:

select 的三大性能瓶颈:

1. fd_set大小限制: FD_SETSIZE默认1024
 - 最多同时处理1024个连接
 - 可以重新定义FD_SETSIZE重编译, 但不推荐
2. 内核态开销: 每次select调用
 - 将整个fd_set从用户态拷贝到内核态
 - 内核需要O(n)遍历所有fd检查状态
 - 返回时修改fd_set再拷贝回用户态
3. 用户态开销: 返回后
 - 应用程序需要O(n)遍历所有fd
 - 找出哪些fd就绪

总复杂度: $O(n)$, n =监控的fd总数
 当 $n=10000$ 时, 每次事件循环开销很大
 当 $n=100000$ 时, select基本不可用

5.2 poll()

工作原理与数据结构

```

// poll消除了fd_set大小限制, 使用动态数组
// 但本质上仍然是O(n)扫描

#include <poll.h>

struct pollfd {
    int    fd;        // 监控的文件描述符
    short  events;     // 请求监控的事件 (输入)
    short  revents;    // 实际发生的事件 (输出, 内核设置)
};

// events/revents的事件类型:
//  POLLIN    数据可读 (包括普通数据和优先级数据)
//  POLLOUT   数据可写 (发送缓冲区有空间)

```

```
// POLLERR  发生错误
// POLLHUP  挂起（对端关闭连接）
// POLLNVAL 无效请求（fd未打开）
// POLLRDHUP 对端关闭连接或关闭写半连接（Linux 2.6.17+）
// POLLPRI  紧急数据可读（带外数据）

int poll(struct pollfd *fds, nfds_t nfds, int timeout);
// timeout:
//   -1: 无限等待（阻塞）
//    0: 立即返回（非阻塞）
//   >0: 等待timeout毫秒
```

完整poll服务器

```
/*
 * poll_echo_server.c - 基于poll的并发回显服务器
 * 编译: gcc -o poll_server poll_echo_server.c
 *
 * 优点: poll没有文件描述符数量限制（使用动态数组）
 * 缺点: 仍然是O(n)遍历，大量连接时效率低
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <poll.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define MAX_CLIENTS 65536    // poll没有FD_SETSIZE限制！
#define BUFFER_SIZE 4096
#define BACKLOG 128

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <port>\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    int port = atoi(argv[1]);

    // 创建监听socket
    int listen_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (listen_fd < 0) { perror("socket"); exit(EXIT_FAILURE); }

    int optval = 1;
    setsockopt(listen_fd, SOL_SOCKET, SO_REUSEADDR, &optval, sizeof(optval));
```

```

struct sockaddr_in server_addr;
memset(&server_addr, 0, sizeof(server_addr));
server_addr.sin_family      = AF_INET;
server_addr.sin_addr.s_addr = INADDR_ANY;
server_addr.sin_port        = htons(port);

if (bind(listen_fd, (struct sockaddr*)&server_addr,
          sizeof(server_addr)) < 0) {
    perror("bind"); exit(EXIT_FAILURE);
}

if (listen(listen_fd, BACKLOG) < 0) {
    perror("listen"); exit(EXIT_FAILURE);
}

printf("Poll-based server listening on port %d\n", port);

// pollfd数组 (poll的核心数据结构)
struct pollfd fds[MAX_CLIENTS];
nfds_t nfds = 0; // 当前数组中有效entry的数量

// 添加监听socket
fds[0].fd      = listen_fd;
fds[0].events  = POLLIN; // 监听读事件 (新连接到达)
nfds = 1;

while (1) {
    // poll: 等待事件
    int ready = poll(fds, nfds, -1); // -1 = 无限等待
    if (ready < 0) {
        if (errno == EINTR) continue;
        perror("poll");
        break;
    }

    // 1. 检查监听socket
    if (fds[0].revents & POLLIN) {
        struct sockaddr_in client_addr;
        socklen_t addr_len = sizeof(client_addr);

        int client_fd = accept(listen_fd,
                                (struct sockaddr*)&client_addr,
                                &addr_len);

        if (client_fd < 0) {
            perror("accept");
        } else if (nfds >= MAX_CLIENTS) {
            fprintf(stderr, "Too many clients\n");
            close(client_fd);
        } else {
            // 将新客户端添加到pollfd数组末尾
            fds[nfds].fd      = client_fd;
            fds[nfds].events  = POLLIN; // 监听读事件
            fds[nfds].revents = 0;
        }
    }
}

```

```

        nfds++;

        char ip[INET_ADDRSTRLEN];
        inet_ntop(AF_INET, &client_addr.sin_addr, ip, sizeof(ip));
        printf("New client: %s:%d (fd=%d, total=%d)\n",
                ip, ntohs(client_addr.sin_port), client_fd, (int)nfds-1);
    }
    ready--;
}

// 2. 遍历所有客户端（跳过监听socket，从索引1开始）
for (nfds_t i = 1; i < nfds && ready > 0; i++) {
    if (fds[i].revents == 0) continue;

    int fd = fds[i].fd;

    // 处理错误和挂起事件
    if (fds[i].revents & (POLLERR | POLLHUP | POLLNVAL)) {
        printf("Client fd=%d error/hangup\n", fd);
        close(fd);
        // 删除：将最后一个元素移到当前位置
        fds[i] = fds[nfds - 1];
        nfds--;
        i--;
        ready--;
        continue;
    }

    // 处理读事件
    if (fds[i].revents & POLLIN) {
        char buffer[BUFFER_SIZE];
        ssize_t n = recv(fd, buffer, sizeof(buffer), 0);

        if (n <= 0) {
            if (n == 0)
                printf("Client fd=%d disconnected\n", fd);
            else
                perror("recv");

            close(fd);
            // 删除：将最后一个元素移到当前位置
            fds[i] = fds[nfds - 1];
            nfds--;
            i--;
        } else {
            // 回显
            send(fd, buffer, n, 0);
        }
        ready--;
    }
}
}

```

```
close(listen_fd);
return 0;
}
```

poll vs select 对比

特性	select	poll
fd数量限制	FD_SETSIZE(1024)	无限制（看内存）
数据结构	fd_set（位图）	struct pollfd（数组）
事件类型	读/写/异常	POLLIN/POLLOUT/POLLERR等
扫描方式	O(n)遍历	O(n)遍历
参数重新传入	每次必须重新设置	复用数组，只重置revents
内核实现	遍历所有fd	遍历所有fd
适用场景	fd数量少(<1000)	fd数量中等(1000-10000)
可移植性	POSIX，几乎所有平台	POSIX

共同缺点：

1. 用户态维护fd列表，内核遍历整个列表 → O(n)
2. 每次调用都需要把fd列表从用户态拷贝到内核态
3. 返回后应用还需要遍历找出就绪fd → 又一次O(n)
4. 随着监控的fd数量增加，性能线性下降

5.3 epoll() ★★★ 最重要

epoll是Linux特有的IO多路复用机制，解决了select/poll的性能问题。

epoll内部工作原理

epoll内核数据结构：



rdllist (Ready List - 就绪链表)	
[epitem_fd5] → [epitem_fd8] → [epitem_fd12] → NULL	
当socket数据到达时：	
1. 网卡中断 → 协议栈处理	
2. 数据放入socket接收队列	
3. 唤醒epoll等待队列上的进程	
4. 回调函数将epitem加入就绪链表	
就绪链表是双向链表，内核直接用链表头返回	
wait_queue_head_t wq ← 等待epoll_wait的进程队列	

epoll事件处理流程：

用户态	内核态
epoll_create(1)	创建eventpoll对象
返回 epfd	初始化红黑树和就绪链表
epoll_ctl(epfd, ADD, fd, &ev)	将fd加入红黑树
	在socket等待队列注册回调 (当socket数据到达时触发)
epoll_wait(epfd, events, max, timeout)	检查就绪链表
如果就绪链表为空：	
进程进入睡眠 (TASK_INTERRUPTIBLE)	
.....	t0时刻，fd=5收到数据
.....	协议栈处理完毕
.....	回调：将fd=5的epitem
.....	加入就绪链表
.....	唤醒等待进程
进程被唤醒	
从就绪链表取出数据	
拷贝events到用户空间	填充epoll_event数组
遍历返回的events	
处理就绪的fd	

epoll三个关键函数

```
#include <sys/epoll.h>

// 1. 创建epoll实例
int epoll_create(int size);
// size: 从Linux 2.6.8后已忽略（只需>0），仅用于向后兼容
// 返回: epoll文件描述符
// 现代用法: int epfd = epoll_create(1);
// 使用完后需要 close(epfd)

// 更好的替代 (Linux 2.6.27+)
int epoll_create1(int flags);
// flags: EPOLL_CLOEXEC - 设置FD_CLOEXEC标志

// 2. 控制epoll监控的fd
int epoll_ctl(int epfd, int op, int fd, struct epoll_event *event);

// op:
//   EPOLL_CTL_ADD: 注册新的fd到epfd中
//   EPOLL_CTL_MOD: 修改已注册fd的监听事件
//   EPOLL_CTL_DEL: 从epfd中删除一个fd

// event结构:
typedef union epoll_data {
    void *ptr;    // 用户自定义指针（最常用，可存储上下文）
    int fd;       // 文件描述符
    uint32_t u32;
    uint64_t u64;
} epoll_data_t;

struct epoll_event {
    uint32_t events; // 监控的事件类型
    epoll_data_t data; // 用户数据
};

// events可设置的值:
//   EPOLLIN    数据可读
//   EPOLLOUT   数据可写
//   EPOLLRDHUP 对端关闭连接或关闭写半连接
//   EPOLLPRI   紧急数据可读
//   EPOLLERR   错误（自动监听，无需手动设置）
//   EPOLLHUP   挂起（自动监听）
//   EPOLLET    边缘触发（Edge Triggered）
//   EPOLLONESHOT 一次性触发

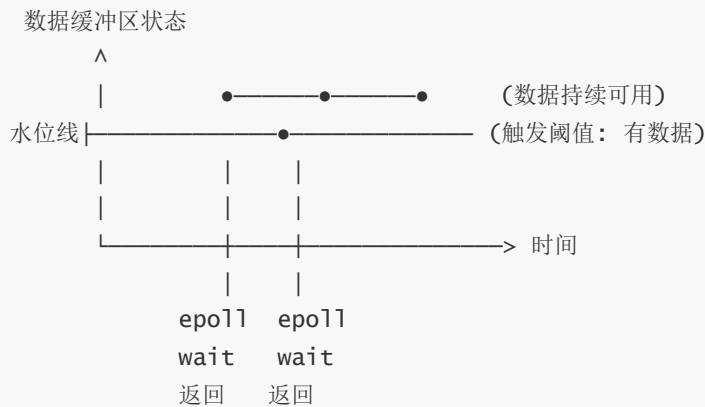
// 3. 等待事件
int epoll_wait(int epfd, struct epoll_event *events,
               int maxevents, int timeout);
// events: 输出参数，存放就绪的事件
// maxevents: 最多返回的事件数
// timeout: -1(阻塞), 0(非阻塞), >0(等待毫秒数)
```

// 返回：就绪的事件数量，0(超时)，-1(错误)

LT (Level Triggered) vs ET (Edge Triggered) ★★★

这是epoll面试中最核心的问题。

水平触发 (Level Triggered) - 默认模式：



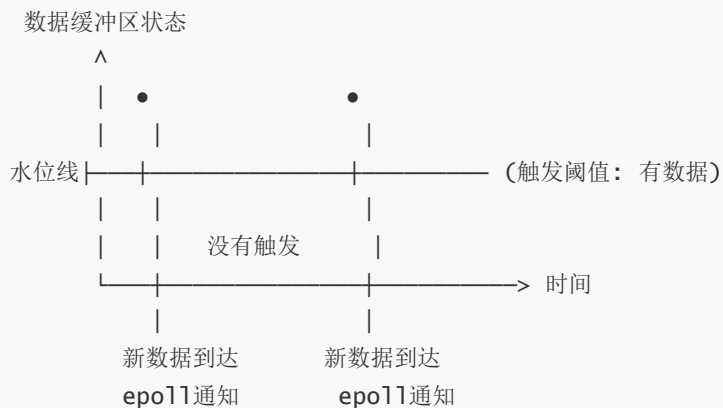
特点：只要缓冲区还有数据，每次epoll_wait都会返回此事件

类比：只要杯子里有水，就一直提醒你喝水

编程：可以只读一部分数据，下次epoll_wait还会通知

风险：如果数据没有读完且不再关注，会陷入"busy loop"

边缘触发 (Edge Triggered)：



特点：只在状态变化时通知一次（从无数据→有数据）

类比：只在杯子空了→有水的这个瞬间提醒你

编程：必须一次性读完所有数据，直到EAGAIN

优势：减少epoll_wait的触发次数

LT与ET编程模式对比：

LT模式（简单）：

```
while (1) {  
    nfd = epoll_wait(epfd, events, MAX_EVENTS, -1);  
    for (i = 0; i < nfd; i++) {  
        if (events[i].events & EPOLLIN) {
```

```

        n = recv(fd, buf, sizeof(buf), 0); // 可以只读一次
        // 如果没读完，下次epoll_wait还会通知
        if (n > 0) process(buf, n);
    }
}
}

```

ET模式（高效但复杂）：

```

while (1) {
    nfds = epoll_wait(epfd, events, MAX_EVENTS, -1);
    for (i = 0; i < nfds; i++) {
        if (events[i].events & EPOLLIN) {
            // 必须循环读，直到返回EAGAIN
            while (1) {
                n = recv(fd, buf, sizeof(buf), 0);
                if (n > 0) {
                    process(buf, n);
                } else if (n < 0) {
                    if (errno == EAGAIN || errno == EWOULDBLOCK) {
                        // 数据已读完，跳出内层循环
                        break;
                    }
                    // 真正的错误
                    handle_error();
                    break;
                } else {
                    // n == 0, 对端关闭
                    handle_close();
                    break;
                }
            }
        }
    }
}
}
}

```

为什么ET模式必须使用非阻塞socket?

原因分析（面试高频题）：

假设ET模式 + 阻塞socket：

1. `epoll_wait`通知fd可读
2. 程序循环调用 `recv(fd, buf, 4096, 0)`
 - 第一次 `recv` 返回 4096 字节
 - 第二次 `recv` 返回 4096 字节
 - 第三次 `recv` 返回 100 字节（缓冲区剩余数据）
 - 第四次 `recv` → 阻塞!!! 因为已经没有数据了
3. 程序永远阻塞在第四次`recv`，无法返回事件循环

不使用非阻塞 + 循环读取 → 会卡死

ET模式 + 非阻塞socket：

1. `epoll_wait`通知fd可读
2. 程序循环调用 `recv(fd, buf, 4096, 0)`

- 第一次 `recv` 返回 4096 字节
- 第二次 `recv` 返回 100 字节
- 第三次 `recv` 返回 -1, `errno = EAGAIN` → 跳出循环

3. 程序返回事件循环, 继续`epoll_wait`

非阻塞保证`recv`不会阻塞, `EAGAIN`作为循环终止条件

EPOLLONESHOT

```
/*
 * EPOLLONESHOT: 确保一个socket连接在同一时刻只被一个线程处理
 *
 * 问题场景（多线程epoll）：
 *   线程A epoll_wait → fd5可读 → 线程A开始处理fd5
 *   线程B epoll_wait → fd5可读（数据还没读完）→ 线程B也开始处理fd5
 *   两个线程同时处理同一个fd → 数据竞争 → 乱序
 *
 * 解决方案：
 *   注册事件时设置 EPOLLONESHOT
 *   一旦事件被触发，该fd自动从epoll监控中移除
 *   处理完毕后，需要重新 EPOLL_CTL_MOD 来恢复监控
 *
 * 注意：即使是ET模式也可能出现多个线程收到同一fd事件的情况
 *       EPOLLONESHOT是专门用来解决这个问题的
 */

// 示例
void epoll_oneshot_example() {
    struct epoll_event ev;
    ev.events = EPOLLIN | EPOLLET | EPOLLONESHOT;
    ev.data.fd = client_fd;
    epoll_ctl(epfd, EPOLL_CTL_ADD, client_fd, &ev);

    // 在处理线程中：
    // ... 处理数据后 ...
    // 重新注册事件，让epoll继续监控此fd
    struct epoll_event ev2;
    ev2.events = EPOLLIN | EPOLLET | EPOLLONESHOT;
    ev2.data.fd = client_fd;
    epoll_ctl(epfd, EPOLL_CTL_MOD, client_fd, &ev2);
}
```

完整epoll ET模式服务器

```
/*
 * epoll_echo_server.c - 基于epoll ET模式的回显服务器
 * 编译: gcc -o epoll_server epoll_echo_server.c
 *
 * 特点：
 *   - 使用ET边缘触发模式
 *   - 所有客户端fd设置为非阻塞
 *   - 循环读取直到EAGAIN
```

```

*    - 性能远超select/poll
*/

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <fcntl.h>
#include <sys/epoll.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define MAX_EVENTS 1024
#define BUFFER_SIZE 4096
#define BACKLOG 128

// 设置文件描述符为非阻塞
int set_nonblocking(int fd) {
    int flags = fcntl(fd, F_GETFL, 0);
    if (flags < 0) {
        perror("fcntl F_GETFL");
        return -1;
    }
    if (fcntl(fd, F_SETFL, flags | O_NONBLOCK) < 0) {
        perror("fcntl F_SETFL");
        return -1;
    }
    return 0;
}

// 创建并绑定监听socket（非阻塞）
int create_listen_socket(int port) {
    int listen_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (listen_fd < 0) {
        perror("socket");
        return -1;
    }

    int optval = 1;
    setsockopt(listen_fd, SOL_SOCKET, SO_REUSEADDR, &optval, sizeof(optval));

    struct sockaddr_in addr;
    memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(port);

    if (bind(listen_fd, (struct sockaddr*)&addr, sizeof(addr)) < 0) {
        perror("bind");
        close(listen_fd);
    }
}

```

```

        return -1;
    }

    if (listen(listen_fd, BACKLOG) < 0) {
        perror("listen");
        close(listen_fd);
        return -1;
    }

    // 设置监听socket为非阻塞
    if (set_nonblocking(listen_fd) < 0) {
        close(listen_fd);
        return -1;
    }

    return listen_fd;
}

// 处理读事件 - ET模式，必须循环读直到EAGAIN
void handle_read(int fd) {
    char buffer[BUFFER_SIZE];

    while (1) {
        ssize_t n = recv(fd, buffer, sizeof(buffer), 0);

        if (n > 0) {
            // 收到数据，回显
            // 注意：写操作在ET模式下也需要循环发送
            // 这里简化处理
            ssize_t sent = 0;
            while (sent < n) {
                ssize_t ret = send(fd, buffer + sent, n - sent, 0);
                if (ret < 0) {
                    if (errno == EAGAIN || errno == EWOULDBLOCK) {
                        // 发送缓冲区满，稍后EPOLLOUT事件会通知
                        // 生产环境需要缓存未发送的数据
                        break;
                    }
                }
                perror("send");
                goto close_conn;
            }
            sent += ret;
        }
        else if (n == 0) {
            // 对端关闭连接
            printf("fd=%d closed by peer\n", fd);
            goto close_conn;
        }
        else {
            if (errno == EAGAIN || errno == EWOULDBLOCK) {
                // 数据已读完，正常退出循环
                break;
            }
        }
        // 被信号中断，继续读
    }
}

```

```

        if (errno == EINTR) {
            continue;
        }
        // 其他错误
        perror("recv");
        goto close_conn;
    }
}
return;

close_conn:
    // 关闭连接前，不需要显式epoll_ctl DEL
    // close会自动从epoll中移除
    close(fd);
}

// 处理新连接
void handle_accept(int listen_fd, int epfd) {
    while (1) {
        struct sockaddr_in client_addr;
        socklen_t addr_len = sizeof(client_addr);

        int client_fd = accept(listen_fd,
                                (struct sockaddr*)&client_addr,
                                &addr_len);

        if (client_fd < 0) {
            if (errno == EAGAIN || errno == EWOULDBLOCK) {
                // 所有连接已处理完毕（ET模式下的边沿触发）
                break;
            }
            perror("accept");
            break;
        }

        char ip[INET_ADDRSTRLEN];
        inet_ntop(AF_INET, &client_addr.sin_addr, ip, sizeof(ip));
        printf("New connection: %s:%d (fd=%d)\n",
              ip, ntohs(client_addr.sin_port), client_fd);

        // 设置客户端socket为非阻塞（ET模式必须）
        set_nonblocking(client_fd);

        // 注册到epoll，使用ET边缘触发
        struct epoll_event ev;
        ev.events = EPOLLIN | EPOLLET; // 边缘触发
        // 也可添加 EPOLLRDHUP | EPOLLOUT（如果需要）
        ev.data.fd = client_fd;

        if (epoll_ctl(epfd, EPOLL_CTL_ADD, client_fd, &ev) < 0) {
            perror("epoll_ctl ADD");
            close(client_fd);
        }
    }
}

```

```

}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <port>\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    int port = atoi(argv[1]);

    int listen_fd = create_listen_socket(port);
    if (listen_fd < 0) {
        exit(EXIT_FAILURE);
    }

    // 创建epoll实例
    int epfd = epoll_create1(0);
    if (epfd < 0) {
        perror("epoll_create1");
        close(listen_fd);
        exit(EXIT_FAILURE);
    }

    // 将监听socket加入epoll
    struct epoll_event ev;
    ev.events = EPOLLIN | EPOLLET; // 监听socket也用ET
    ev.data.fd = listen_fd;
    if (epoll_ctl(epfd, EPOLL_CTL_ADD, listen_fd, &ev) < 0) {
        perror("epoll_ctl: listen_fd");
        close(listen_fd);
        close(epfd);
        exit(EXIT_FAILURE);
    }

    struct epoll_event events[MAX_EVENTS];

    printf("Epoll (ET mode) server listening on port %d\n", port);

    // 事件循环
    while (1) {
        int nfd = epoll_wait(epfd, events, MAX_EVENTS, -1);
        if (nfd < 0) {
            if (errno == EINTR) continue;
            perror("epoll_wait");
            break;
        }

        for (int i = 0; i < nfd; i++) {
            int fd = events[i].data.fd;

            if (fd == listen_fd) {
                // 新连接
                handle_accept(listen_fd, epfd);
            }
        }
    }
}

```

```

        } else {
            // 客户端数据
            if (events[i].events & (EPOLLERR | EPOLLHUP)) {
                // 错误或挂起
                printf("fd=%d error/hangup\n", fd);
                close(fd);
            } else if (events[i].events & EPOLLIN) {
                handle_read(fd);
            }
        }
    }
}

close(listen_fd);
close(epfd);
return 0;
}

```

epoll性能分析

为什么epoll比select/poll快?

1. $O(1)$ 的就绪事件获取
 select/poll: 返回后应用需要遍历所有fd $\rightarrow O(n)$
 epoll: 内核返回的就是就绪event数组 $\rightarrow O(1)$ per event
2. 事件驱动 vs 轮询
 select/poll: 每次调用都要遍历所有fd $\rightarrow$ 类似"轮询"
 epoll: socket数据到达时通过回调自动加入就绪链表
 $\rightarrow$ "事件驱动", 不需要扫描全部fd
3. 内存拷贝减少
 select/poll: 每次调用都要把整个fd列表拷入内核 $\rightarrow O(n)$ 内存拷贝
 epoll: epoll_ctl只在添加/删除时一次性传入 $\rightarrow O(1)$ 后续调用
4. 红黑树的增删效率
 epoll_ctl: $O(\log N)$
 select: 每次重新设置所有fd
 poll: 每次遍历数组
5. mmap加速 (部分实现)
 内核和用户空间共享内存, 减少数据拷贝

性能对比 (理论值):

连接数	select	poll	epoll
10	$\sim 1\mu s$	$\sim 1\mu s$	$\sim 1\mu s$
100	$\sim 10\mu s$	$\sim 10\mu s$	$\sim 2\mu s$
1000	$\sim 100\mu s$	$\sim 100\mu s$	$\sim 5\mu s$
10000	$\sim 1000\mu s$	$\sim 1000\mu s$	$\sim 10\mu s$
100000	不可用	$\sim 10ms$	$\sim 50\mu s$

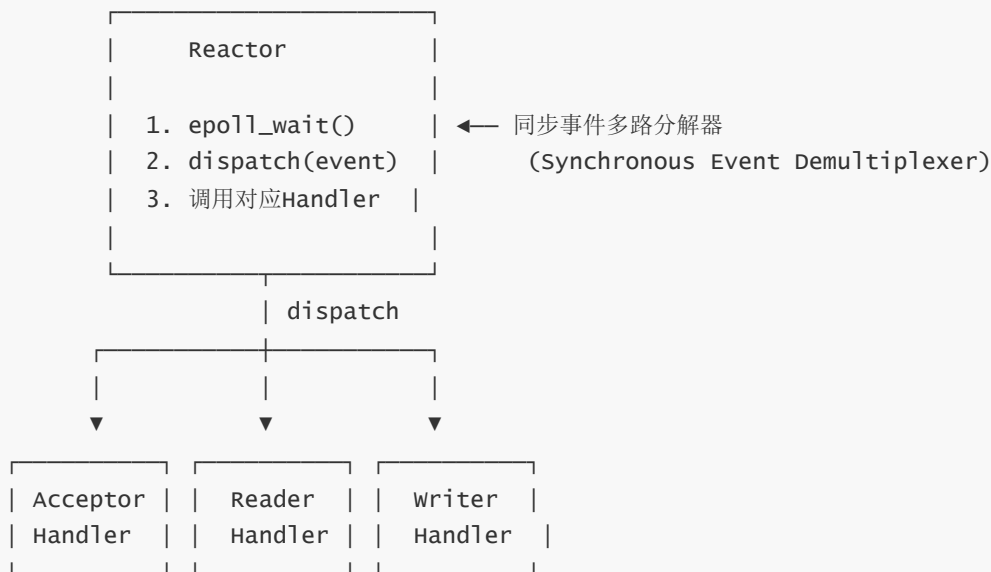


6. Reactor模型 ★★★

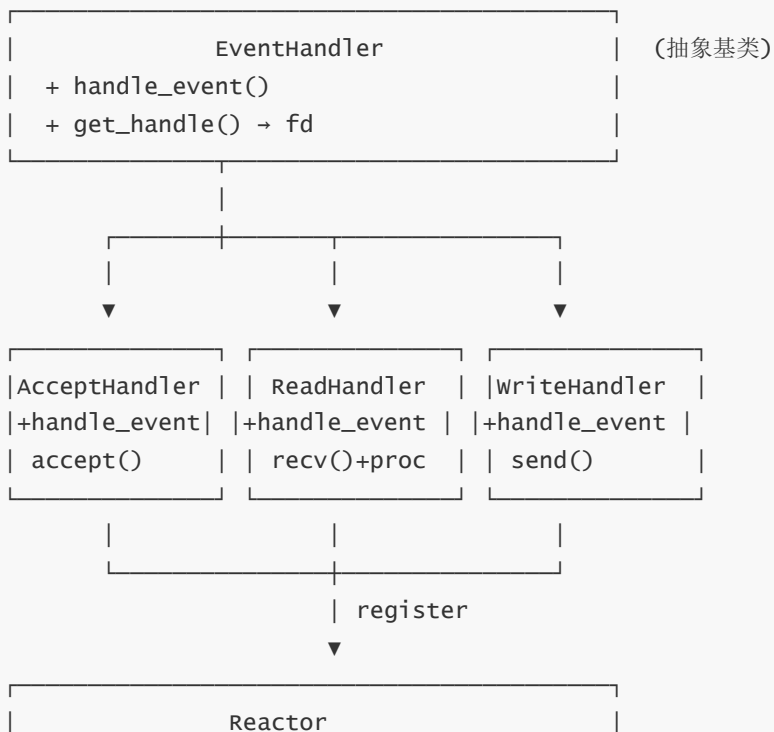
Reactor（反应堆）是一种事件驱动的IO多路复用设计模式。

6.1 Reactor核心思想

Reactor模式 = IO多路复用 + 事件分发 + 事件处理



Reactor类图：



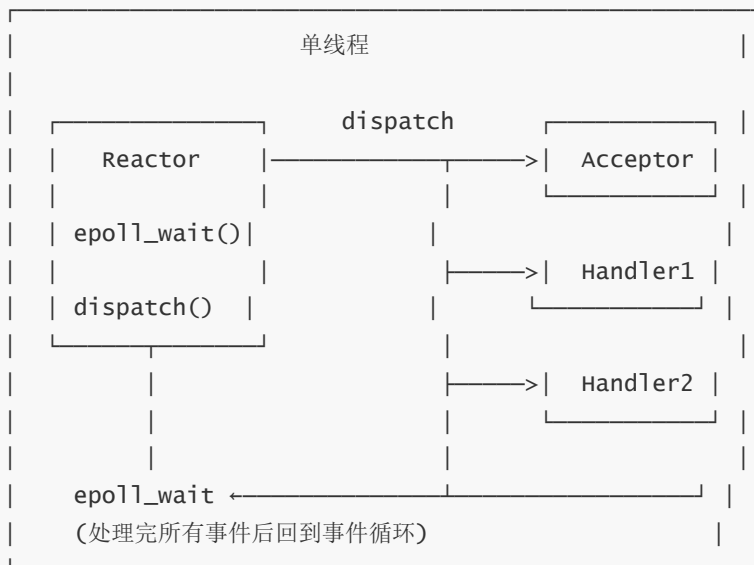
```

| + register_handler(handler) |
| + remove_handler(handler) |
| + handle_events() ← 事件循环 |
| - epoll_fd |
| - handlers: map<int, EventHandler*> |

```

6.2 单Reactor单线程

单Reactor单线程模型：



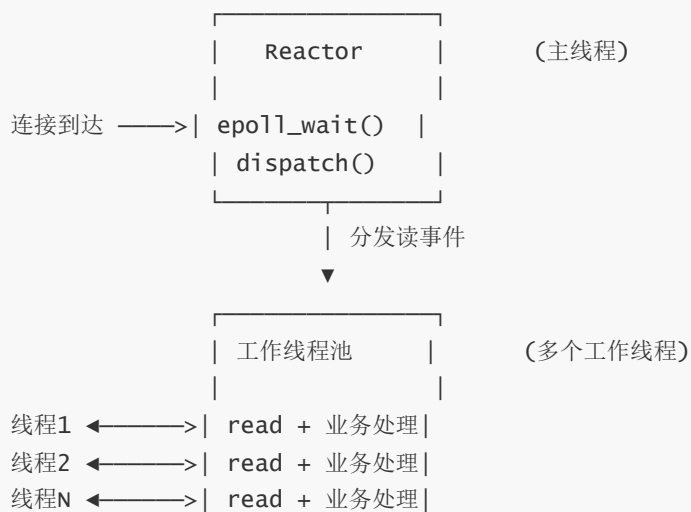
代表实现：Redis 6.0之前、Node.js事件循环

优点：简单，无锁，无上下文切换

缺点：一个Handler阻塞会影响所有连接，无法利用多核CPU

6.3 单Reactor多线程

单Reactor多线程模型：





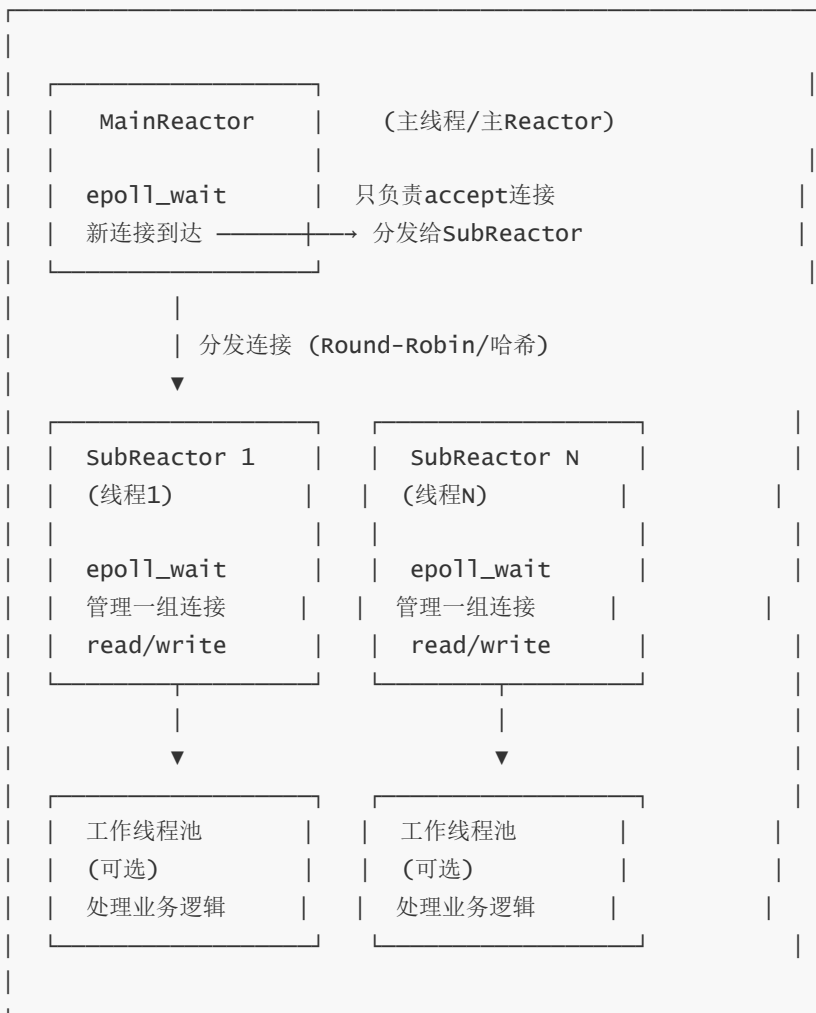
代表实现：大多数Java NIO应用

优点：利用多核CPU处理业务逻辑

缺点：Reactor单线程可能成为瓶颈(accept + IO读写在主线程)
线程间需要同步

6.4 多Reactor多线程 (Master-Worker / Main-Sub) ★★★

多Reactor多线程模型（最主流的Reactor模型）：



代表实现：Netty, Nginx, Memcached

优点：

1. 主Reactor只负责accept，不负责读写
2. 子Reactor各自负责一组连接的IO，可以充分利用多核

3. 连接均匀分配到各个子Reactor，负载均衡
4. 一个子Reactor阻塞不会影响其他子Reactor

6.5 完整Reactor服务器示例

```
/*
 * reactor_server.c - 简化的Reactor模式服务器
 * 编译: gcc -o reactor_server reactor_server.c
 *
 * 架构: 主Reactor(accept) + 子Reactor(read/write) + 线程池(业务处理)
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <fcntl.h>
#include <pthread.h>
#include <sys/epoll.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define MAX_EVENTS      1024
#define BUFFER_SIZE     4096
#define BACKLOG         128
#define MAX_THREADS     4
#define SUB_REACTOR_NUM 2

// ===== 事件处理接口 =====

// 前向声明
struct event_handler;

// 事件类型
enum event_type {
    EVENT_READ   = EPOLLIN,
    EVENT_WRITE  = EPOLLOUT,
    EVENT_ERROR  = EPOLLERR,
};

// 事件处理器接口
typedef void (*event_callback)(struct event_handler *handler,
                               enum event_type type);

typedef struct event_handler {
    int fd;
    event_callback callback;
    void *user_data; // 用户自定义数据
} event_handler_t;
```

```
// ===== Reactor =====

typedef struct reactor {
    int epfd;
    int running;
    pthread_t thread;
} reactor_t;

// Reactor: 注册事件
int reactor_register(reactor_t *r, event_handler_t *handler,
                    uint32_t events) {
    struct epoll_event ev;
    ev.events = events;
    ev.data.ptr = handler; // epoll_data_t的ptr指向handler
    return epoll_ctl(r->epfd, EPOLL_CTL_ADD, handler->fd, &ev);
}

// Reactor: 修改事件
int reactor_modify(reactor_t *r, event_handler_t *handler,
                  uint32_t events) {
    struct epoll_event ev;
    ev.events = events;
    ev.data.ptr = handler;
    return epoll_ctl(r->epfd, EPOLL_CTL_MOD, handler->fd, &ev);
}

// Reactor: 移除事件
int reactor_remove(reactor_t *r, event_handler_t *handler) {
    return epoll_ctl(r->epfd, EPOLL_CTL_DEL, handler->fd, NULL);
}

// Reactor: 事件循环 (核心)
void reactor_event_loop(reactor_t *r) {
    struct epoll_event events[MAX_EVENTS];

    while (r->running) {
        int nfds = epoll_wait(r->epfd, events, MAX_EVENTS, -1);
        if (nfds < 0) {
            if (errno == EINTR) continue;
            perror("epoll_wait");
            break;
        }

        for (int i = 0; i < nfds; i++) {
            event_handler_t *handler =
                (event_handler_t*)events[i].data.ptr;

            uint32_t ev = events[i].events;

            // 错误优先处理
            if (ev & (EPOLLERR | EPOLLHUP)) {
                handler->callback(handler, EVENT_ERROR);
            }
        }
    }
}
```

```

        continue;
    }

    if (ev & EPOLLIN) {
        handler->callback(handler, EVENT_READ);
    }

    if (ev & EPOLLOUT) {
        handler->callback(handler, EVENT_WRITE);
    }
}
}

// 创建Reactor
reactor_t* reactor_create() {
    reactor_t *r = (reactor_t*)calloc(1, sizeof(reactor_t));
    r->epfd = epoll_create1(0);
    r->running = 1;
    return r;
}

void reactor_destroy(reactor_t *r) {
    r->running = 0;
    close(r->epfd);
    free(r);
}

// ===== 连接上下文 =====

typedef struct connection {
    int fd;
    char buffer[BUFFER_SIZE];
    size_t buf_len;
    reactor_t *reactor; // 所属的子Reactor
    event_handler_t handler;
} connection_t;

// ===== 子Reactor的IO回调 =====

// 读回调
void connection_read_callback(event_handler_t *handler,
                              enum event_type type) {
    connection_t *conn = (connection_t*)handler->user_data;
    char buffer[BUFFER_SIZE];

    ssize_t n = recv(conn->fd, buffer, sizeof(buffer), 0);
    if (n > 0) {
        // 回显（实际项目中这里应该提交到线程池处理）
        send(conn->fd, buffer, n, 0);
    } else if (n == 0) {
        // 对端关闭
        printf("Connection fd=%d closed\n", conn->fd);
    }
}

```

```

        reactor_remove(conn->reactor, handler);
        close(conn->fd);
        free(conn);
    } else {
        if (errno != EAGAIN && errno != EWOULDBLOCK) {
            perror("recv");
            reactor_remove(conn->reactor, handler);
            close(conn->fd);
            free(conn);
        }
    }
}

// ===== 主Reactor的Accept回调 =====

// 子Reactor列表（用于分发连接）
typedef struct {
    reactor_t *sub_reactors[SUB_REACTOR_NUM];
    int round_robin_index;
} acceptor_data_t;

static int set_nonblocking(int fd) {
    int flags = fcntl(fd, F_GETFL, 0);
    return fcntl(fd, F_SETFL, flags | O_NONBLOCK);
}

void accept_callback(event_handler_t *handler, enum event_type type) {
    acceptor_data_t *acceptor = (acceptor_data_t*)handler->user_data;

    while (1) {
        struct sockaddr_in client_addr;
        socklen_t addr_len = sizeof(client_addr);

        int client_fd = accept(handler->fd,
                               (struct sockaddr*)&client_addr,
                               &addr_len);

        if (client_fd < 0) {
            if (errno == EAGAIN || errno == EWOULDBLOCK) break;
            perror("accept");
            break;
        }

        printf("Accepted fd=%d\n", client_fd);
        set_nonblocking(client_fd);

        // 创建连接对象
        connection_t *conn = (connection_t*)calloc(1, sizeof(connection_t));
        conn->fd = client_fd;

        // Round-Robin选择子Reactor
        int idx = acceptor->round_robin_index % SUB_REACTOR_NUM;
        acceptor->round_robin_index++;
        conn->reactor = acceptor->sub_reactors[idx];
    }
}

```

```

        // 注册读事件
        conn->handler.fd = client_fd;
        conn->handler.callback = connection_read_callback;
        conn->handler.user_data = conn;

        reactor_register(conn->reactor, &conn->handler, EPOLLIN);
    }
}

// ===== 主函数 =====

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <port>\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    int port = atoi(argv[1]);

    // 创建监听socket
    int listen_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (listen_fd < 0) { perror("socket"); exit(EXIT_FAILURE); }

    int optval = 1;
    setsockopt(listen_fd, SOL_SOCKET, SO_REUSEADDR, &optval, sizeof(optval));
    set_nonblocking(listen_fd);

    struct sockaddr_in server_addr;
    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(port);

    if (bind(listen_fd, (struct sockaddr*)&server_addr,
             sizeof(server_addr)) < 0) {
        perror("bind"); exit(EXIT_FAILURE);
    }

    if (listen(listen_fd, BACKLOG) < 0) {
        perror("listen"); exit(EXIT_FAILURE);
    }

    // ---- 创建子Reactor ----
    reactor_t *sub_reactors[SUB_REACTOR_NUM];
    for (int i = 0; i < SUB_REACTOR_NUM; i++) {
        sub_reactors[i] = reactor_create();
    }

    // ---- 创建主Reactor ----
    reactor_t *main_reactor = reactor_create();

    // Acceptor数据结构（包含子Reactor列表）

```

```

acceptor_data_t acceptor;
for (int i = 0; i < SUB_REACTOR_NUM; i++) {
    acceptor.sub_reactors[i] = sub_reactors[i];
}
acceptor.round_robin_index = 0;

// 在主Reactor上注册accept事件
event_handler_t accept_handler;
accept_handler.fd = listen_fd;
accept_handler.callback = accept_callback;
accept_handler.user_data = &acceptor;

reactor_register(main_reactor, &accept_handler, EPOLLIN);

printf("Reactor server listening on port %d (MainReactor + %d SubReactors)\n",
       port, SUB_REACTOR_NUM);

// ---- 启动子Reactor线程 ----

// 线程函数：运行子Reactor的事件循环
// 简化：这里在主线程中运行所有Reactor
// 生产环境应该为每个子Reactor创建线程

// 简化版本：直接在同一个线程中模拟
// （实际项目中，每个子Reactor在独立的线程中运行reactor_event_loop）

// 主线程运行主Reactor和所有子Reactor的事件循环（这里只是演示）
// 实际使用时，每个Reactor应该在独立线程中调用reactor_event_loop
reactor_event_loop(main_reactor);

// 清理
reactor_destroy(main_reactor);
for (int i = 0; i < SUB_REACTOR_NUM; i++) {
    reactor_destroy(sub_reactors[i]);
}
close(listen_fd);

return 0;
}

```

6.6 事件循环设计要点

```

/*
 * 事件循环（Event Loop）核心设计：
 *
 * 1. 单层循环 vs 多层循环
 *
 * 单层事件循环（Node.js, Redis）：
 * while (running) {
 *     nfds = epoll_wait(...)
 *     for (event in events) {
 *         handler(event)
 *     }
 * }

```

```

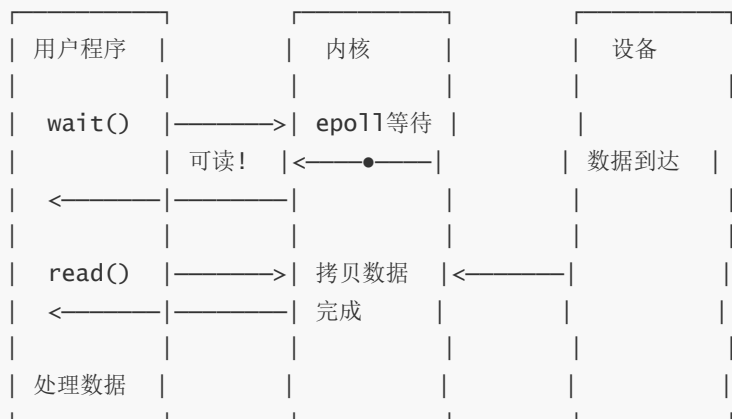
*     }
* }
*
* 多层事件循环 (Nginx, Netty):
* 主循环:
*     while (running) {
*         nfds = epoll_wait(main_epfd, ...) // 只处理accept
*         for (event in events) {
*             accept + dispatch_to_sub_reactor
*         }
*     }
*
* 子循环(每个线程):
*     while (running) {
*         nfds = epoll_wait(sub_epfd, ...) // 处理IO读写
*         for (event in events) {
*             read/write + process
*         }
*     }
*
*
* 2. 定时器集成
* - 在epoll_wait中设置timeout参数
* - 维护最小堆定时器队列
* - epoll_wait超时后处理到期的定时器
*
* 3. 优雅退出
* - 设置running标志
* - 使用eventfd唤醒epoll_wait
* - 清理所有连接
* - 等待pending操作完成
*/

```

7. Proactor模型

7.1 Reactor vs Proactor

Reactor (同步IO):

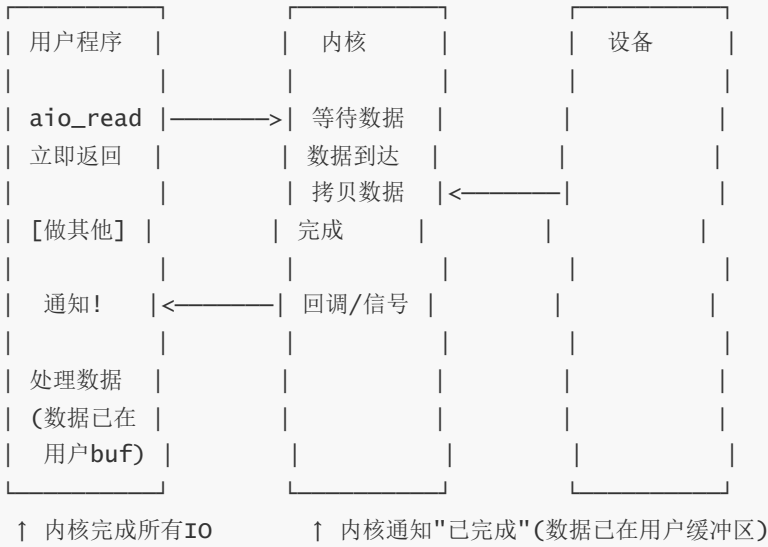


↑ 用户自己调用read

↑ 内核只通知"可读"

用户还需要自己拷贝数据

Proactor (异步IO):



关键区别:

	Reactor	Proactor
IO操作	用户代码执行recv/send	内核执行全部IO操作
主动方	用户主动读取	内核主动完成
通知时机	IO就绪时通知	IO完成时通知
数据位置	通知时数据在内核缓冲区	通知时数据已在用户缓冲区
实现方式	epoll/select/poll	IOCP(win) / io_uring(Linux)
编程难度	相对简单	相对复杂
性能	好	更好（理论上）
平台	跨平台	平台相关

7.2 IOCP (Windows)

Windows的IOCP (I/O Completion Port) 是经典的Proactor实现:

```
// windows IOCP 伪代码
// 1. 创建完成端口
HANDLE iocp = CreateIoCompletionPort(INVALID_HANDLE_VALUE, NULL, 0, 0);

// 2. 绑定socket到完成端口
CreateIoCompletionPort((HANDLE)socket, iocp, completion_key, 0);

// 3. 发起异步操作
WSARecv(socket, &buf, 1, &bytes, &flags, &overlapped, NULL);
// 注意: 不等待结果, 立即返回!

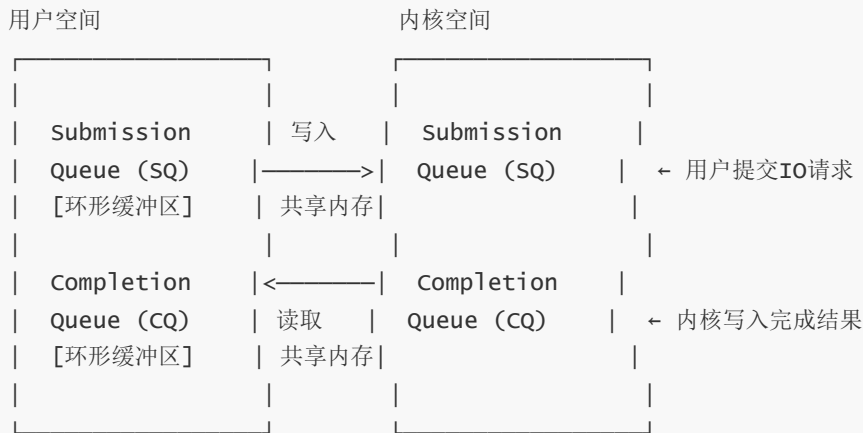
// 4. 等待完成通知
while (true) {
    GetQueuedCompletionStatus(iocp, &bytes, &key, &overlapped, INFINITE);
    // 操作已完成, 数据已经在buf中
    // 处理数据...
```

```
}
```

7.3 io_uring (Linux 5.1+)

io_uring是Linux对Proactor模式的实现，通过共享内存环形缓冲区实现零拷贝的异步IO。

io_uring架构：



特点：

1. SQ和CQ是用户态和内核态共享的内存区域
2. 用户提交请求只需写入SQ，无需系统调用（提交模式）
3. 内核完成请求后写入CQ
4. 用户从CQ读取完成结果
5. 可以使用IORING_SETUP_SQPOLL让内核线程轮询SQ，进一步减少系统调用

```
// io_uring 简单示例
#include <liburing.h>

void io_uring_example() {
    struct io_uring ring;
    io_uring_queue_init(256, &ring, 0);

    // 提交异步读请求
    struct io_uring_sqe *sqe = io_uring_get_sqe(&ring);
    io_uring_prep_read(sqe, fd, buf, size, offset);
    io_uring_submit(&ring);

    // 等待完成
    struct io_uring_cqe *cqe;
    io_uring_wait_cqe(&ring, &cqe);
    // 此时数据已在buf中
    if (cqe->res > 0) {
        process(buf, cqe->res);
    }
    io_uring_cqe_seen(&ring, cqe);

    io_uring_queue_exit(&ring);
}
```

8. 常见Socket问题

8.1 Address already in use (SO_REUSEADDR)

```
/*
 * 问题: bind() 失败: Address already in use
 *
 * 原因: 上次运行的服务端主动关闭连接后, 处于TIME_WAIT状态 (2MSL ≈ 60秒)
 *       在这个时间内, 端口仍被占用, 无法重新bind
 *
 * 解决方法:
 */

// 方法1: 在bind之前设置SO_REUSEADDR (最常用)
int reuse = 1;
setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &reuse, sizeof(reuse));

// 方法2: 设置SO_REUSEPORT (多进程/线程绑定同一端口)
int reuseport = 1;
setsockopt(sockfd, SOL_SOCKET, SO_REUSEPORT, &reuseport, sizeof(reuseport));

// 方法3: 调整内核参数 (不推荐主动关闭服务器端连接)
// echo 1 > /proc/sys/net/ipv4/tcp_tw_reuse

/*
 * SO_REUSEADDR 到底做了什么?
 *   - 允许绑定处于TIME_WAIT状态的地址
 *   - 允许多个socket绑定到同一端口 (需要IP不同, 或使用SO_REUSEPORT)
 *   - 注意: 不能绑定处于ESTABLISHED状态的地址
 */
```

8.2 Broken Pipe (SIGPIPE)

```
/*
 * 问题: 向已关闭的socket写入数据
 *   - 第一次write → 对端RST
 *   - 第二次write → 内核发送SIGPIPE信号 → 进程默认终止!
 *
 * 解决方法:
 */

// 方法1: 忽略SIGPIPE信号 (最常用)
signal(SIGPIPE, SIG_IGN);

// 方法2: 使用MSG_NOSIGNAL标志
send(fd, buf, len, MSG_NOSIGNAL); // 返回EPIPE错误而不是发信号

// 方法3: 使用SO_NOSIGPIPE (macOS/BSD)
int val = 1;
setsockopt(fd, SOL_SOCKET, SO_NOSIGPIPE, &val, sizeof(val));
```

8.3 Connection Reset by Peer

```
/*
 * "Connection reset by peer" (ECONNRESET)
 *
 * 可能原因：
 * 1. 对端进程崩溃（没有正常close，而是exit，OS发送RST）
 * 2. 对端调用close时SO_LINGER设置为立即关闭（发送RST而不是FIN）
 * 3. 对端关闭了连接，但本端还在发送数据
 * 4. 对端收到了不期望的数据（如向已关闭的连接发数据）
 *
 * 处理方式：
 *   recv返回-1，errno=ECONNRESET
 *   关闭本端socket，清理资源
 */
```

8.4 部分发送/接收 (Partial send/recv)

```
/*
 * TCP是字节流，send/recv不保证一次发送/接收完所有数据
 *
 * 正确处理方式：
 */

// 可靠发送：循环发送直到所有数据发送完毕
ssize_t reliable_send(int fd, const void *buf, size_t len) {
    size_t total = 0;
    while (total < len) {
        ssize_t n = send(fd, (const char*)buf + total, len - total, 0);
        if (n < 0) {
            if (errno == EINTR) continue;
            if (errno == EPIPE) {
                // 连接已断开
                return -1;
            }
            return -1;
        }
        total += n;
    }
    return total;
}

// 可靠接收：循环接收直到收到指定长度
ssize_t reliable_recv(int fd, void *buf, size_t len) {
    size_t total = 0;
    while (total < len) {
        ssize_t n = recv(fd, (char*)buf + total, len - total, 0);
        if (n < 0) {
            if (errno == EINTR) continue;
            return -1;
        }
    }
}
```

```

        if (n == 0) {
            // 对端关闭连接
            return total; // 返回已收到字节数
        }
        total += n;
    }
    return total;
}

// 对于变长协议：先读长度头，再读数据体
// 协议格式：[4字节长度(网络序)][数据体]
int recv_message(int fd, char **data, uint32_t *len) {
    // 1. 先读4字节长度
    uint32_t net_len;
    ssize_t n = reliable_recv(fd, &net_len, 4);
    if (n != 4) return -1;

    *len = ntohl(net_len);

    // 2. 再读数据体
    *data = (char*)malloc(*len);
    if (*data == NULL) return -1;

    n = reliable_recv(fd, *data, *len);
    if (n != (ssize_t)*len) {
        free(*data);
        return -1;
    }

    return 0;
}

```

8.5 非阻塞connect

```

/*
 * 非阻塞connect用于：
 *   1. 设置连接超时（阻塞connect无法设置超时）
 *   2. 在connect的同时处理其他事情
 */

int nonblocking_connect(const char *ip, int port, int timeout_ms) {
    int sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd < 0) return -1;

    // 设为非阻塞
    int flags = fcntl(sockfd, F_GETFL, 0);
    fcntl(sockfd, F_SETFL, flags | O_NONBLOCK);

    struct sockaddr_in addr;
    memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_port = htons(port);
}

```

```

inet_pton(AF_INET, ip, &addr.sin_addr);

// 发起非阻塞connect
int ret = connect(sockfd, (struct sockaddr*)&addr, sizeof(addr));
if (ret == 0) {
    // 居然立即成功（连接到localhost时可能发生）
    // 恢复阻塞模式
    fcntl(sockfd, F_SETFL, flags);
    return sockfd;
}

if (errno != EINPROGRESS) {
    // 真正的错误
    close(sockfd);
    return -1;
}

// connect正在进行中，用select/poll等待完成
fd_set wfds;
FD_ZERO(&wfds);
FD_SET(sockfd, &wfds);

struct timeval tv;
tv.tv_sec = timeout_ms / 1000;
tv.tv_usec = (timeout_ms % 1000) * 1000;

ret = select(sockfd + 1, NULL, &wfds, NULL, &tv);
if (ret <= 0) {
    // 超时或错误
    close(sockfd);
    return -1;
}

// select返回，检查连接是否成功
int error = 0;
socklen_t len = sizeof(error);
if (getsockopt(sockfd, SOL_SOCKET, SO_ERROR, &error, &len) < 0) {
    close(sockfd);
    return -1;
}

if (error != 0) {
    // 连接失败
    errno = error;
    close(sockfd);
    return -1;
}

// 连接成功，恢复阻塞模式
fcntl(sockfd, F_SETFL, flags);
return sockfd;
}

```

8.6 优雅关闭 (Graceful Shutdown)

```
/*
 * 优雅关闭的四次挥手（主动关闭方）
 *
 * 正确做法：
 */
void graceful_shutdown(int fd) {
    // 1. 关闭写端 → 发送FIN
    shutdown(fd, SHUT_WR);

    // 2. 继续读取对端的数据
    char buf[4096];
    while (1) {
        ssize_t n = recv(fd, buf, sizeof(buf), 0);
        if (n <= 0) {
            break; // n==0: 收到对端的FIN
                  // n<0: 错误
        }
        // 处理剩余数据...
    }

    // 3. 完全关闭
    close(fd);
}

/*
 * SO_LINGER 选项：
 *
 * struct linger {
 *     int l_onoff; // 0=关闭, 非0=开启
 *     int l_linger; // 延迟时间(秒)
 * };
 *
 * 情况1: l_onoff = 0 (默认)
 *     close()立即返回, 内核在后台发送缓冲区数据并FIN
 *
 * 情况2: l_onoff != 0, l_linger = 0
 *     close()立即返回, 发送RST而不是FIN, 丢弃缓冲区数据
 *     → 导致对端收到ECONNRESET而不是正常关闭
 *
 * 情况3: l_onoff != 0, l_linger > 0
 *     close()阻塞最多l_linger秒
 *     等待数据发送完毕和FIN确认
 *     超时则发送RST
 */

// SO_LINGER 示例
struct linger ling;
ling.l_onoff = 1;
ling.l_linger = 5; // 最多等5秒
setsockopt(fd, SOL_SOCKET, SO_LINGER, &ling, sizeof(ling));
```

9. 面试总结

9.1 面试重点

知识点	重要度	说明
TCP三次握手/四次挥手	★★★★★	必问，必须能画状态图
epoll工作原理	★★★★★	红黑树+就绪链表，必须讲清楚
LT vs ET	★★★★★	区别、使用场景、为什么ET需要非阻塞
Reactor vs Proactor	★★★★★	大厂面试几乎必问
select/poll/epoll区别	★★★★★	数据结构、性能、限制
Socket API	★★★★★	常用函数参数和返回值
TIME_WAIT	★★★★★	产生原因、影响、解决方法
阻塞/非阻塞/异步IO	★★★★★	五种IO模型，能画出流程图
close vs shutdown	★★★★	区别、适用场景
多Reactor多线程	★★★★	架构图、为什么这么设计

9.2 高频考点

- 1. **epoll为什么比select快？** → 事件驱动 vs 轮询、O(1) vs O(n)、红黑树、就绪链表、mmap
- 2. **LT和ET的区别？ET为什么必须非阻塞？** → 触发次数、缓冲区残留、阻塞风险
- 3. **什么是Reactor？什么是Proactor？区别？** → 谁调用IO读写、通知时机、数据位置
- 4. **TCP状态转换（三次握手、四次挥手）** → 能画出完整状态图
- 5. **TIME_WAIT产生原因，有什么影响？** → 2MSL、被动关闭方、SO_REUSEADDR
- 6. **listen的backlog参数是什么意思？** → SYN队列、ACCEPT队列

9.3 常见陷阱

- 1. **select每次都要重新设置fd_set** → 因为select会修改传入的fd_set
- 2. **ET模式读数据必须读到EAGAIN** → 否则可能永远收不到新通知
- 3. **父进程fork后必须close(client_fd)** → 否则引用计数不归零
- 4. **send/recv不保证一次收发完毕** → 需要循环处理
- 5. **SIGPIPE默认终止进程** → 必须忽略或使用MSG_NOSIGNAL
- 6. **sockaddr_in需要memset清零** → 特别是sin_zero字段
- 7. **端口号和IP地址要用网络字节序** → htons/htonl
- 8. **epoll的event.data.ptr** → 比data.fd更好，可以携带上下文

9.4 建议背诵内容

1. `epoll_create` / `epoll_ctl` / `epoll_wait` 的函数签名和参数
2. `select` / `poll` / `epoll` 的区别表格
3. LT vs ET 的区别和代码模式
4. TCP三次握手/四次挥手的状态转换图
5. 五种IO模型的名称和特点
6. Reactor三种模型的架构描述
7. 完整`epoll ET`服务器代码结构 (至少能写出框架)
8. `select`的`fd_set`操作宏: `FD_ZERO`/`FD_SET`/`FD_CLR`/`FD_ISSET`